

Optimization for Deep Learning

Choose the data · remember direction and scale · control the step

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Backprop measured the slope; today we decide how to move

$\theta_t \rightarrow \text{forward} \rightarrow \mathcal{L} \rightarrow \text{backward} \rightarrow g_t \rightarrow \text{optimizer} \rightarrow \theta_{t+1}$

Backprop measured the slope; today we decide how to move

$\theta_t \rightarrow \text{forward} \rightarrow \mathcal{L} \rightarrow \text{backward} \rightarrow g_t \rightarrow \text{optimizer} \rightarrow \theta_{t+1}$

LECTURES 1–4

defined the loss, model, computation graph, and gradient g_t .

LECTURE 5

chooses the data, remembers useful history, and sets the step $\Delta\theta_t$.

$$\theta_t \rightarrow \textbf{forward} \rightarrow \mathcal{L} \rightarrow \textbf{backward} \rightarrow g_t \rightarrow \textbf{optimizer} \rightarrow \theta_{t+1}$$

LECTURES 1–4

defined the loss, model, computation graph, and gradient g_t .

LECTURE 5

chooses the data, remembers useful history, and sets the step $\Delta\theta_t$.

The gradient is a measurement. An optimizer turns it into $\Delta\theta_t := \theta_{t+1} - \theta_t$.

Outline

1. **Choose the data per update** — full batch, one example, or a minibatch
2. **Understand the estimate** — unbiased centre, variance, epochs, and updates
3. **Expose a geometric failure** — why an exact gradient can still zigzag and crawl
4. **Add useful memory** — EWMA $\rightarrow$ momentum $\rightarrow$ RMSProp $\rightarrow$ Adam
5. **Finish the training loop** — decay, PyTorch, and constrained parameters

Three examples will keep the mechanisms concrete

A • FOUR-POINT LINE FIT

$\hat{y} = b + wx$ on
 $(-1, -1), (0, 1), (1, 3), (2, 6)$.

Use: full/SGD/
minibatch,
unbiasedness, variance,
decay.

B • SCALED-FEATURE REGRESSION

Four two-feature
examples with one
feature $10 \times$ larger.

Use: ravines, EWMA,
momentum, RMSProp,
Adam.

C • TINY XOR NETWORK

The familiar Lecture 2
XOR task and a $2 \rightarrow 4 \rightarrow$
1 MLP.

Use: verify the
mechanisms in an actual
neural-network loop.

Every contour, arrow, and trajectory is computed from a displayed objective. The notebooks reproduce the underlying calculations and seeded experiments.

**How much data should define
one update?**

What optimization means in this course

For scalar-output examples $(\mathbf{x}_i, y_i)$ and all trainable parameters $\boldsymbol{\theta}$,

$$\hat{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i), \quad \ell_{i(\boldsymbol{\theta})} = \ell(\hat{y}_i, y_i), \quad \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\boldsymbol{\theta})}.$$

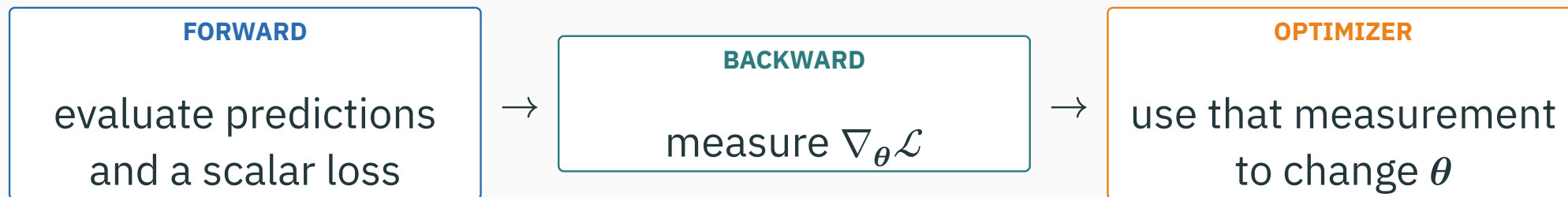
Course convention from L1–L3: bold lowercase/uppercase symbols are vectors/matrices; $y_i, \hat{y}_i, \ell_i$, and $\mathcal{L}$ are scalars.

What optimization means in this course

For scalar-output examples $(\mathbf{x}_i, y_i)$ and all trainable parameters $\boldsymbol{\theta}$,

$$\hat{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i), \quad \ell_{i(\boldsymbol{\theta})} = \ell(\hat{y}_i, y_i), \quad \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\boldsymbol{\theta})}.$$

Course convention from L1–L3: bold lowercase/uppercase symbols are vectors/matrices; $y_i, \hat{y}_i, \ell_i$, and $\mathcal{L}$ are scalars.

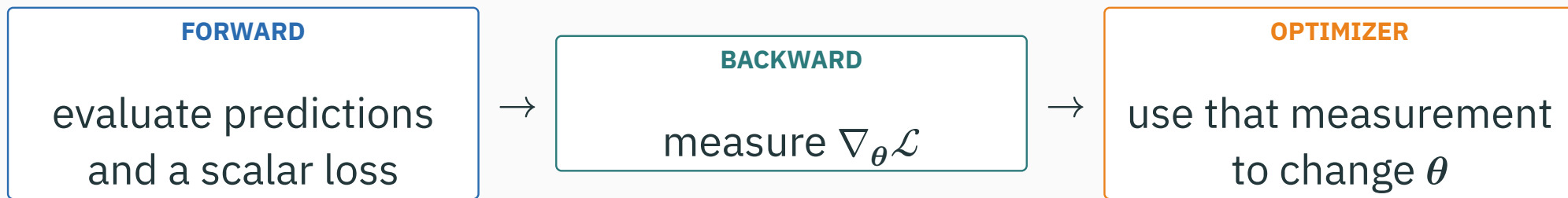


What optimization means in this course

For scalar-output examples $(\mathbf{x}_i, y_i)$ and all trainable parameters $\boldsymbol{\theta}$,

$$\hat{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i), \quad \ell_{i(\boldsymbol{\theta})} = \ell(\hat{y}_i, y_i), \quad \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\boldsymbol{\theta})}.$$

Course convention from L1–L3: bold lowercase/uppercase symbols are vectors/matrices; $y_i, \hat{y}_i, \ell_i$, and $\mathcal{L}$ are scalars.



Today the objective stays fixed. We change which examples measure its gradient and how the optimizer transforms that measurement.

Revision: vanilla gradient descent is a four-line loop

For N examples and parameter vector θ ,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\theta)}.$$

Revision: vanilla gradient descent is a four-line loop

For N examples and parameter vector θ ,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\theta)}.$$

evaluate	$\mathcal{L}(\theta_t)$
differentiate	$g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$
choose η	$\eta > 0$
update	$\theta_{t+1} = \theta_t - \eta g_t$

Revision: vanilla gradient descent is a four-line loop

For N examples and parameter vector θ ,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\theta)}.$$

evaluate	$\mathcal{L}(\theta_t)$
differentiate	$g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$
choose η	$\eta > 0$
update	$\theta_{t+1} = \theta_t - \eta g_t$

Batch gradient descent (also called **full-batch GD**) uses all N examples for every update. “Vanilla GD” describes the update rule; the batch specifies which gradient it receives.

Example A: compute one full-batch gradient

Use $\hat{y}_i = b + wx_i$ and $\ell_i = \frac{1}{2}(\hat{y}_i - y_i)^2$. Then

$$\nabla_{\theta} \ell_i = (\hat{y}_i - y_i)(1, x_i).$$

Example A: compute one full-batch gradient

Use $\hat{y}_i = b + wx_i$ and $\ell_i = \frac{1}{2}(\hat{y}_i - y_i)^2$. Then

$$\nabla_{\boldsymbol{\theta}} \ell_i = (\hat{y}_i - y_i)(1, x_i).$$

At $\boldsymbol{\theta}_0 = (b, w) = (2, -1)$, the prediction becomes $\hat{y}_i = 2 - x_i$:

x_i	y_i	$\hat{y}_i = 2 - x_i$	$\hat{y}_i - y_i$	$\nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_0)$
-1	-1	$2 - (-1) = 3$	$3 - (-1) = 4$	$(4, -4)$
0	1	$2 - 0 = 2$	$2 - 1 = 1$	$(1, 0)$
1	3	$2 - 1 = 1$	$1 - 3 = -2$	$(-2, -2)$
2	6	$2 - 2 = 0$	$0 - 6 = -6$	$(-6, -12)$

Read each row left to right: predict $\hat{y}_i$, form the residual $\hat{y}_i - y_i$, then multiply by $(1, x_i)$ to obtain the gradient.

$$\mathbf{g}_{\text{full}} = \frac{1}{4}[(4, -4) + (1, 0) + (-2, -2) + (-6, -12)] = (-0.75, -4.5).$$

$$\mathbf{g}_{\text{full}} = \frac{1}{4}[(4, -4) + (1, 0) + (-2, -2) + (-6, -12)] = (-0.75, -4.5).$$

PARAMETER UPDATE

$$\boldsymbol{\theta}_1 = (2, -1) - 0.1(-0.75, -4.5) = (2.075, -0.55)$$

PRODUCED LINE

$$f_{\boldsymbol{\theta}_1}(x) = 2.075 - 0.55x$$

$$\mathbf{g}_{\text{full}} = \frac{1}{4}[(4, -4) + (1, 0) + (-2, -2) + (-6, -12)] = (-0.75, -4.5).$$

PARAMETER UPDATE

$$\boldsymbol{\theta}_1 = (2, -1) - 0.1(-0.75, -4.5) = (2.075, -0.55)$$

PRODUCED LINE

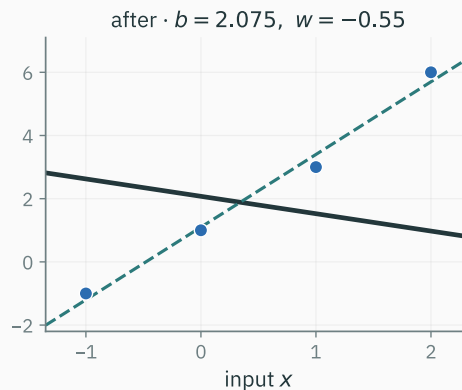
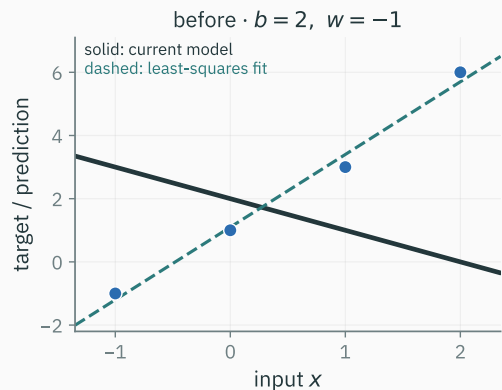
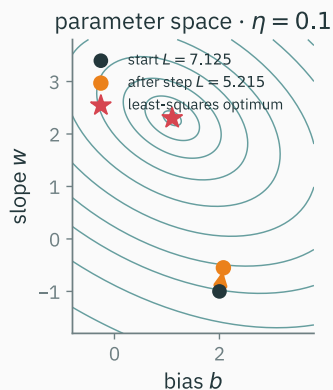
$$f_{\boldsymbol{\theta}_1}(x) = 2.075 - 0.55x$$

NEW RESIDUALS $r_{1,i} = f_{\boldsymbol{\theta}_1}(x_i) - y_i, \quad \mathbf{r}_1 = (3.625, 1.075, -1.475, -5.025)$

Because $\ell_i = \frac{1}{2}r_i^2$ and $N = 4$, the full loss is $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{8}\|\mathbf{r}\|_2^2$.

before step	$\mathcal{L}(\boldsymbol{\theta}_0) = \frac{1}{8}(4^2 + 1^2 + (-2)^2 + (-6)^2) = 7.125$
after step	$\mathcal{L}(\boldsymbol{\theta}_1) = \frac{1}{8}(3.625^2 + 1.075^2 + (-1.475)^2 + (-5.025)^2) \approx 5.215$

One gradient step changes the fitted model



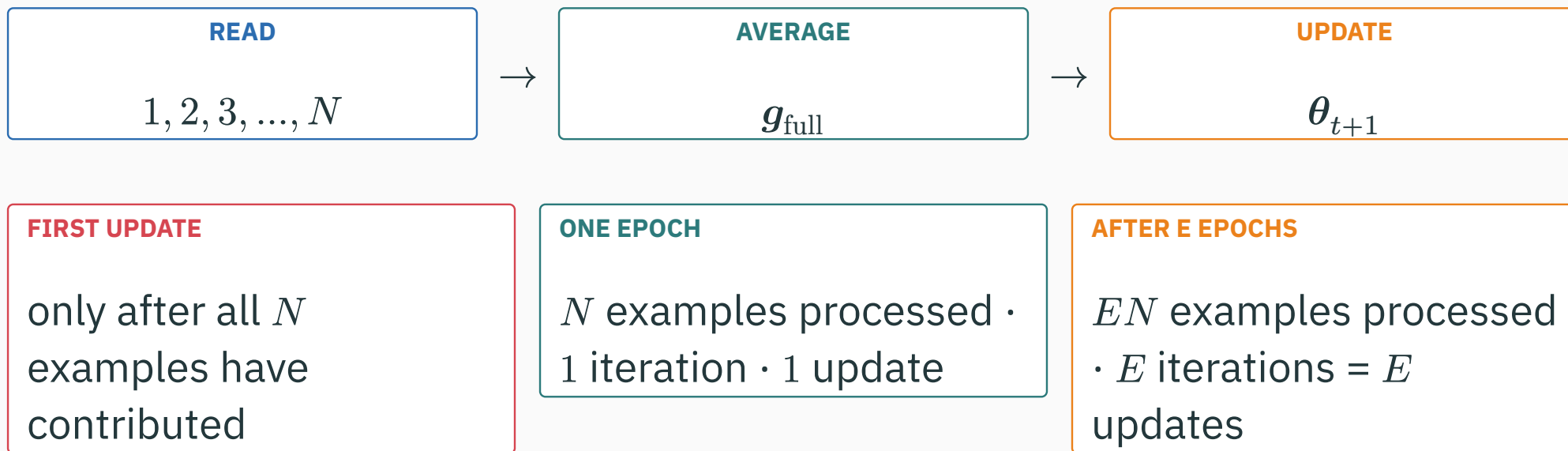
COMPUTED · THE FOUR DISPLAYED EXAMPLES · NOTEBOOK 00

| The contour point and the fitted line are two views of the same parameters (b, w) .

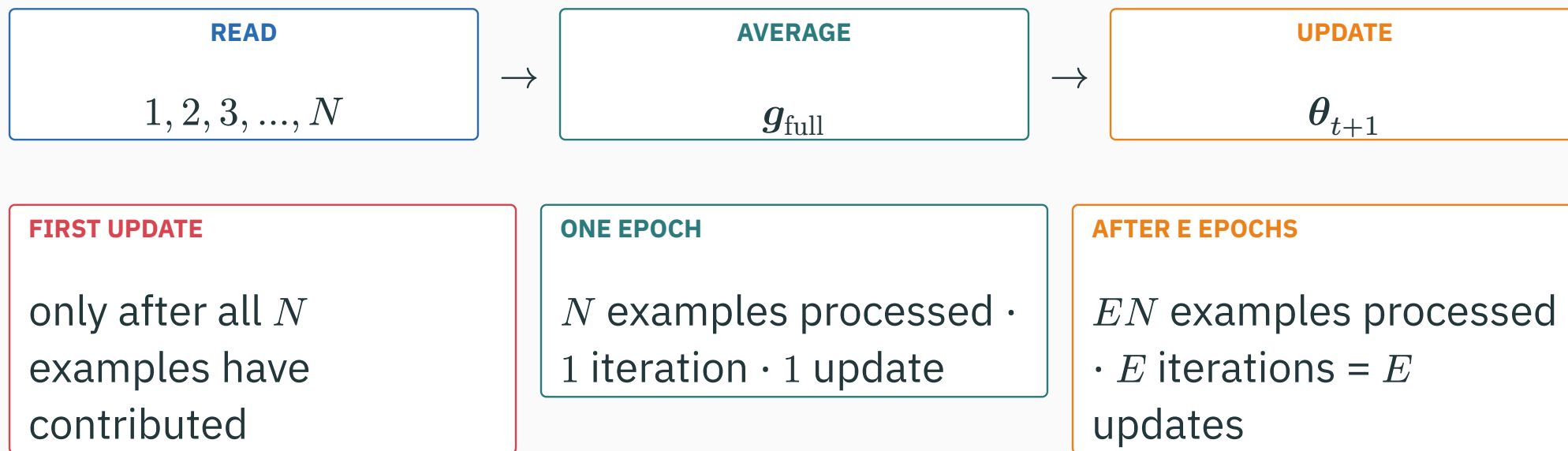
Batch gradient descent is exact—but every update must wait



Batch gradient descent is exact—but every update must wait



Batch gradient descent is exact—but every update must wait



Work versus time: one full-batch update processes N examples. Parallel hardware changes wall-clock time, but the parameters still wait for the reduction over all N contributions.

The training clock: examples, iterations, updates, epochs

N examples, $1 \leq B \leq N$ per batch $\rightarrow K = \lceil \frac{N}{B} \rceil$ iterations per epoch

$$\underbrace{\{3, 1\}}_{\text{iteration 1}} \quad \underbrace{\{4, 2\}}_{\text{iteration 2}}$$

Example: $N = 4, B = 2$. Both minibatches together process every example once—one epoch.

The training clock: examples, iterations, updates, epochs

D DERIVATION

N examples, $1 \leq B \leq N$ per batch $\rightarrow K = \lceil \frac{N}{B} \rceil$ iterations per epoch

$\underbrace{\{3, 1\}}_{\text{iteration 1}} \quad \underbrace{\{4, 2\}}_{\text{iteration 2}}$

Example: $N = 4, B = 2$. Both minibatches together process every example once—one epoch.

EPOCH one pass through N

ITERATION one minibatch loop body

UPDATE one parameter change

The training clock: examples, iterations, updates, epochs

N examples, $1 \leq B \leq N$ per batch $\rightarrow K = \lceil \frac{N}{B} \rceil$ iterations per epoch

$\underbrace{\{3, 1\}}_{\text{iteration 1}} \quad \underbrace{\{4, 2\}}_{\text{iteration 2}}$

Example: $N = 4, B = 2$. Both minibatches together process every example once—one epoch.

EPOCH one pass through N

ITERATION one minibatch loop body

UPDATE one parameter change

method	B	examples / iteration	iterations / epoch	updates / epoch
full batch	N	N	1	1
SGD	1	1	N	N
minibatch	B	B	$\lceil \frac{N}{B} \rceil$	$\lceil \frac{N}{B} \rceil$

Normally, iteration = update. After E epochs: EN examples and $E \lceil \frac{N}{B} \rceil$ updates. Gradient accumulation or skipped steps are exceptions.

FULL-BATCH GD

$$\mathbf{g}_t = \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t)$$

Exact for the current model—but all N contributions must arrive before one update.

SGD

$$I_t \sim \text{Uniform}\{1, \dots, N\}$$

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \ell_{I_t}(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{g}}_t$$

FULL-BATCH GD

$$\mathbf{g}_t = \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t)$$

Exact for the current model—but all N contributions must arrive before one update.

SGD

$$I_t \sim \text{Uniform}\{1, \dots, N\}$$

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \ell_{I_t}(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{g}}_t$$

Stochastic means the selected index I_t is random. The model and loss stay fixed; only the example used to estimate their gradient changes.

FULL-BATCH GD

$$\mathbf{g}_t = \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t)$$

Exact for the current model—but all N contributions must arrive before one update.

SGD

$$I_t \sim \text{Uniform}\{1, \dots, N\}$$

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \ell_{I_t}(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{g}}_t$$

Stochastic means the selected index I_t is random. The model and loss stay fixed; only the example used to estimate their gradient changes.

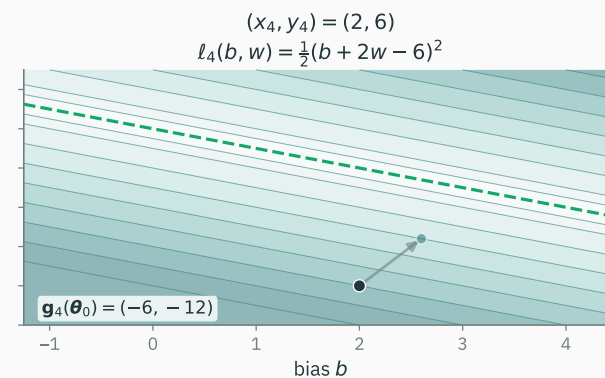
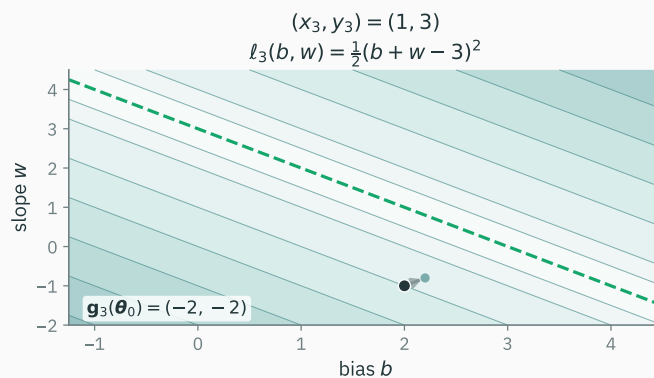
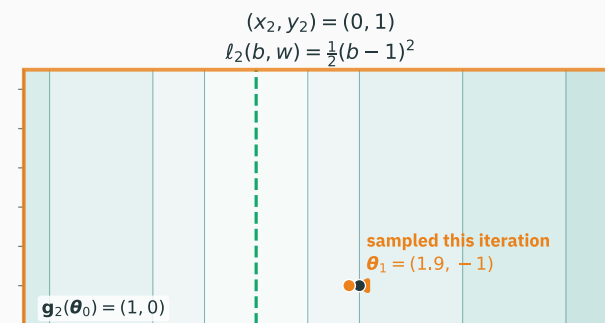
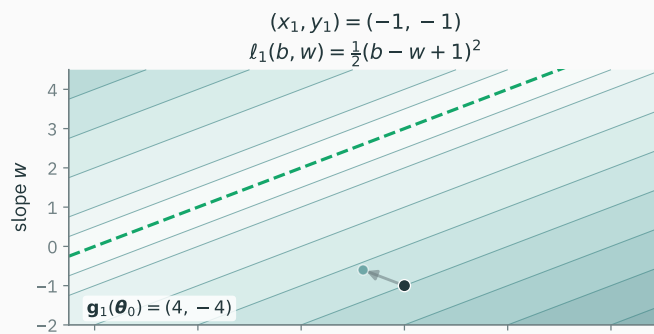
If $I_0 = 2$, then $(x_2, y_2) = (0, 1)$ and

$$\ell_2(b, w) = \frac{1}{2}(b - 1)^2, \quad \hat{\mathbf{g}}_0 = (2 - 1)(1, 0) = (1, 0), \quad \boldsymbol{\theta}_1 = (1.9, -1).$$

This step follows ℓ_2 . We have not yet shown what it does to the average loss $\mathcal{L}$.

Each example proposes a different update

$$\ell_i(b, w) = \frac{1}{2}(b + wx_i - y_i)^2, \quad \mathcal{L}(b, w) = \frac{1}{4} \sum_{i=1}^4 \ell_i(b, w).$$



COMPUTED · ORANGE = SAMPLED $I_0 = 2$ · MUTED ARROWS = ALTERNATIVES · FULL BATCH AVERAGES ALL FOUR

Worked check: ℓ_2 decreases, but $\mathcal{L}$ increases

D DERIVATION

1 One draw chooses the step

draw $I_0 = 2: (x_2, y_2) = (0, 1)$
 $\hat{y}_2 = 2 + (-1)(0) = 2$

$\hat{\mathbf{g}}_0 = (2 - 1)(1, 0) = (1, 0)$

$\boldsymbol{\theta}_1 = (2, -1) - 0.1(1, 0)$
 $= (1.9, -1)$

sampled $\ell_2: 0.500 \rightarrow 0.405 \downarrow$

$i = 1 \quad x = -1$

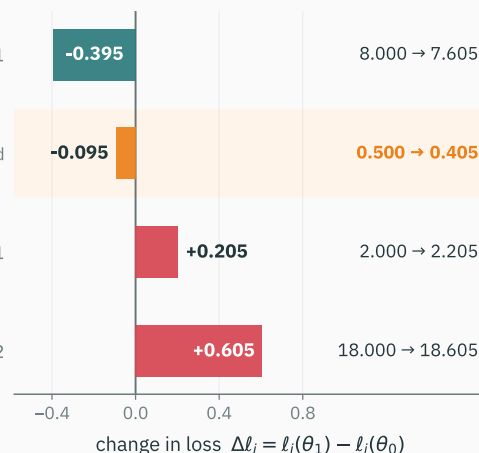
$i = 2 \quad x = 0$ sampled

$i = 3 \quad x = 1$

$i = 4 \quad x = 2$

2 Re-score all four examples

before $\rightarrow$ after



3 Average the changes

sampled: $\Delta \ell_2 = -0.095$

other three: $+0.415$

$$\sum_i \Delta \ell_i = +0.320$$

$$\Delta \mathcal{L} = +0.320/4 = +0.080$$

full $\mathcal{L}: 7.125 \rightarrow 7.205 \uparrow$

COMPUTED • ORANGE POINT SAMPLED • SAME FOUR-EXAMPLE FULL LOSS

The step is correct for ℓ_2 : $\Delta \ell_2 = -0.095$. Across all four examples, $\sum_i \Delta \ell_i = +0.320$, so $\Delta \mathcal{L} = +\frac{0.320}{4} = +0.080$.

A minibatch is the useful middle ground

For minibatch $\mathcal{B}_t$, let $B_t = |\mathcal{B}_t|$ (the final batch may be smaller):

$$\hat{\mathbf{g}}_t = \frac{1}{B_t} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t).$$

A minibatch is the useful middle ground

For minibatch $\mathcal{B}_t$, let $B_t = |\mathcal{B}_t|$ (the final batch may be smaller):

$$\hat{\mathbf{g}}_t = \frac{1}{B_t} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t).$$

If $\mathcal{B} = \{1, 2\}$ contains the first two gradients in our table,

$$\hat{\mathbf{g}} = \frac{1}{2}[(4, -4) + (1, 0)] = (2.5, -2).$$

For minibatch $\mathcal{B}_t$, let $B_t = |\mathcal{B}_t|$ (the final batch may be smaller):

$$\hat{\mathbf{g}}_t = \frac{1}{B_t} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t).$$

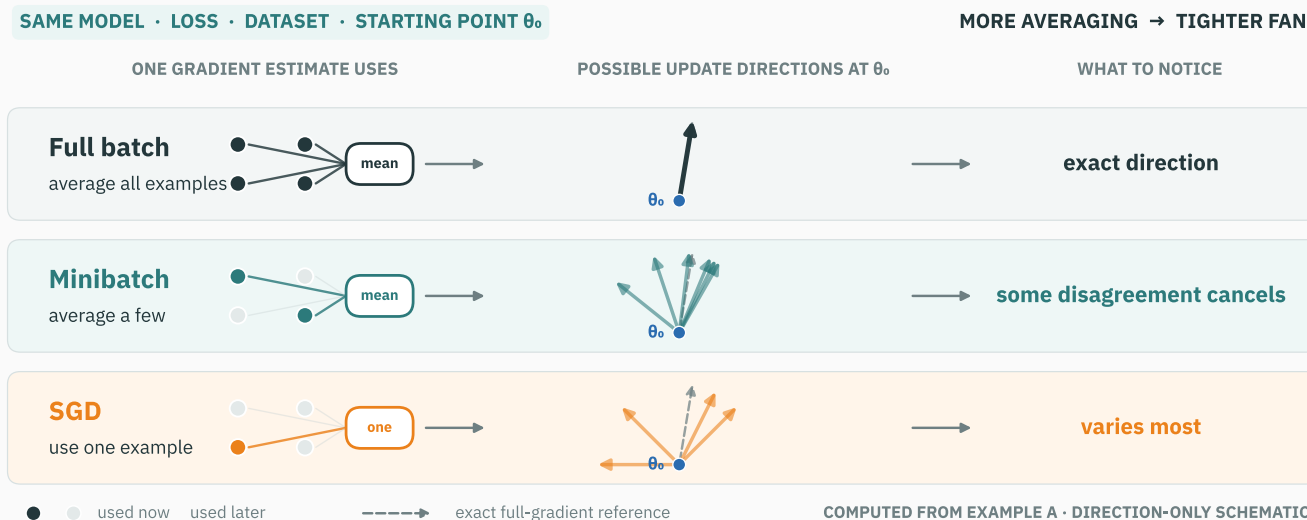
If $\mathcal{B} = \{1, 2\}$ contains the first two gradients in our table,

$$\hat{\mathbf{g}} = \frac{1}{2}[(4, -4) + (1, 0)] = (2.5, -2).$$

$$\boldsymbol{\theta}_1 = (2, -1) - 0.1(2.5, -2) = (1.75, -0.8).$$

Batch size is one knob: $B_t = 1$ is **SGD**; $B_t = N$ is **full-batch GD**. Next: **what does averaging buy us?**

Why minibatches help: averaging cancels some disagreement



COMPUTED FROM EXAMPLE A · FIXED MODEL, DATA, AND PARAMETERS · ONLY THE AVERAGING CHANGES

One example can pull in an idiosyncratic direction. Averaging a few cancels some disagreement; averaging all examples leaves the exact full-data direction.

Batch, stochastic, and minibatch GD—qualitatively

	Full batch	SGD	Minibatch
examples / update	N	1	B
updates / epoch	1	N	$\lceil \frac{N}{B} \rceil$
gradient noise	none	highest	intermediate
parallelism	high but large	poor	usually efficient
near optimum; fixed $\eta > 0$	smooth	typically wanders if sampling noise remains	less wandering as B grows

In deep learning, “SGD” often informally means the whole minibatch-SGD family. The batch size states which member is actually used.

Choosing the batch size in PyTorch

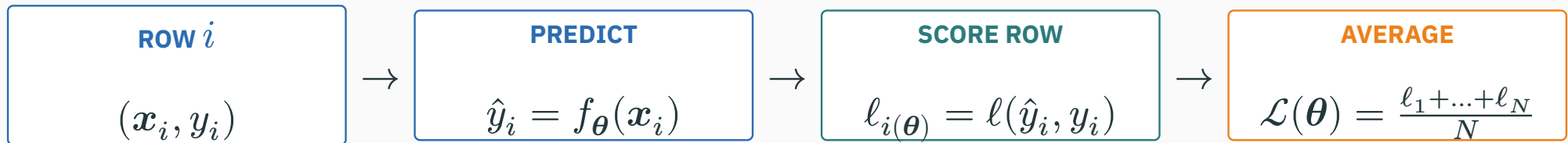
```
loader = DataLoader(dataset, batch_size=B, shuffle=True)

for epoch in range(E):
    for xb, yb in loader:          # one iteration / minibatch
        optimizer.zero_grad(set_to_none=True)
        pred = model(xb)
        loss = 0.5 * (pred - yb).square().mean()
        loss.backward()           # gradient of this batch mean
        optimizer.step()          # one parameter update
```

`torch.optim.SGD` transforms the gradient supplied by the loop; its name does **not** imply $B = 1$. The factor $\frac{1}{2}$ matches our analytic half-MSE convention.

**How can one example estimate
the full gradient?**

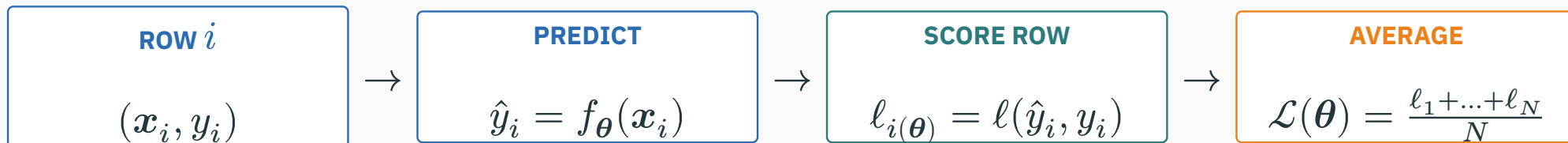
From data rows to the full loss



For Example A,

$$\ell_{i(b,w)} = \frac{1}{2}(b + wx_i - y_i)^2, \quad \mathcal{L}(b, w) = \frac{1}{4} \sum_{i=1}^4 \ell_{i(b,w)}.$$

From data rows to the full loss



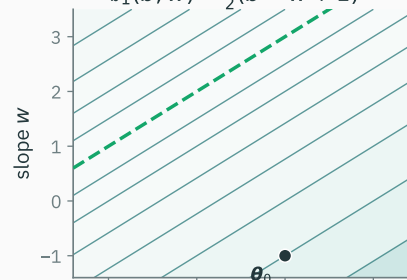
For Example A,

$$\ell_{i(b,w)} = \frac{1}{2}(b + wx_i - y_i)^2, \quad \mathcal{L}(b, w) = \frac{1}{4} \sum_{i=1}^4 \ell_{i(b,w)}.$$

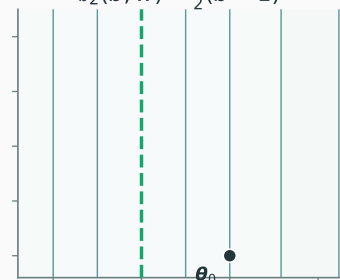
The full loss is an average of per-example losses. Therefore its gradient is the same average: $\nabla \mathcal{L} = \frac{1}{N} \sum_i \nabla \ell_i$.

Each example defines its own loss surface

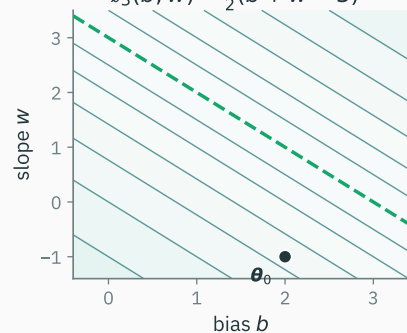
Example 1: $(x_1, y_1) = (-1, -1)$
 $\ell_1(b, w) = \frac{1}{2}(b - w + 1)^2$



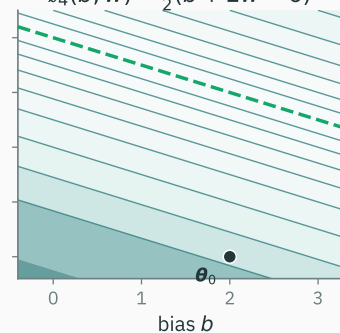
Example 2: $(x_2, y_2) = (0, 1)$
 $\ell_2(b, w) = \frac{1}{2}(b - 1)^2$



Example 3: $(x_3, y_3) = (1, 3)$
 $\ell_3(b, w) = \frac{1}{2}(b + w - 3)^2$



Example 4: $(x_4, y_4) = (2, 6)$
 $\ell_4(b, w) = \frac{1}{2}(b + 2w - 6)^2$

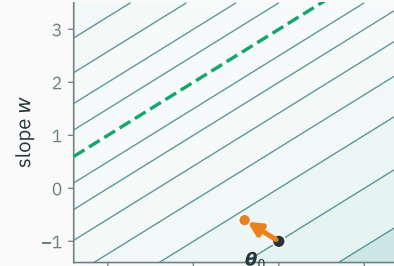


same model $\theta_0 = (2, -1)$ in every panel · dashed green: zero-loss line for that one example

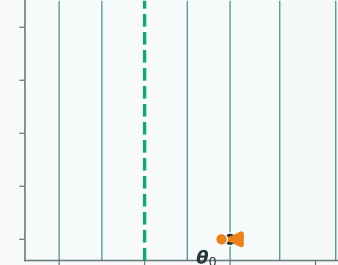
COMPUTED FROM THE FOUR DISPLAYED EXAMPLE A ROWS · IDENTICAL AXES AND CONTOUR LEVELS

Same parameters, different descent directions

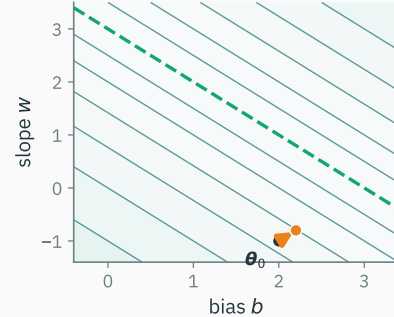
Example 1: $\ell_1(b, w) = \frac{1}{2}(b - w + 1)^2$
 $\Delta\theta_1 = -\eta\nabla\ell_1 = (-0.40, +0.40)$



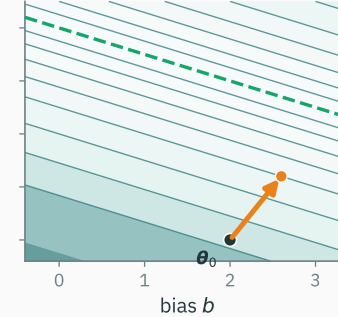
Example 2: $\ell_2(b, w) = \frac{1}{2}(b - 1)^2$
 $\Delta\theta_2 = -\eta\nabla\ell_2 = (-0.10, +0.00)$



Example 3: $\ell_3(b, w) = \frac{1}{2}(b + w - 3)^2$
 $\Delta\theta_3 = -\eta\nabla\ell_3 = (+0.20, +0.20)$



Example 4: $\ell_4(b, w) = \frac{1}{2}(b + 2w - 6)^2$
 $\Delta\theta_4 = -\eta\nabla\ell_4 = (+0.60, +1.20)$

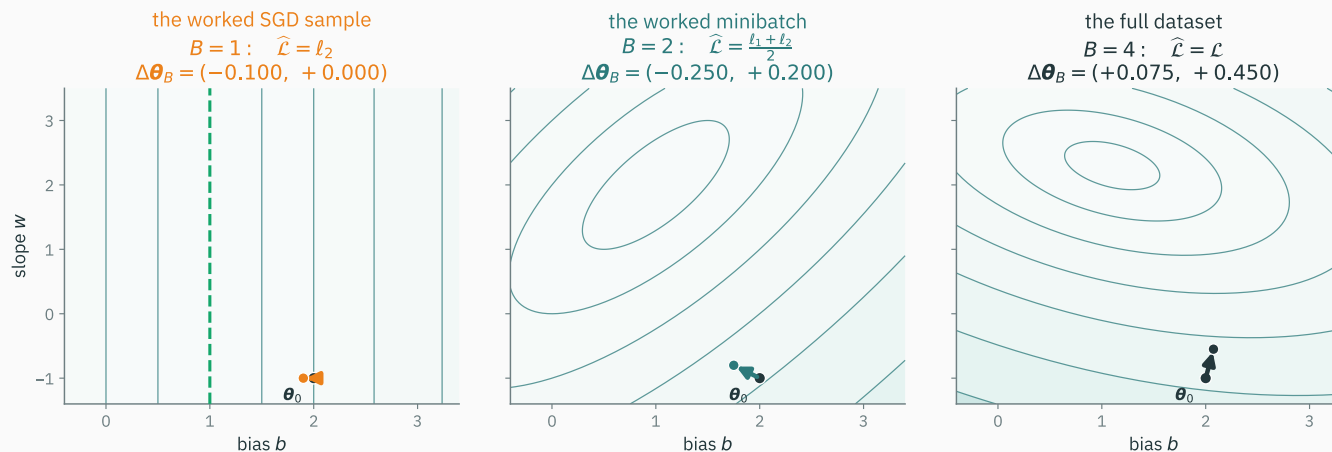


orange: the exact parameter update for that panel, with $\eta = 0.10$ · coordinates are $(\Delta b, \Delta w)$

COMPUTED FROM EXAMPLE A · SAME $\theta_0 = (2, -1)$ AND $\eta = 0.1$ IN EVERY PANEL

Each arrow is the **update** $\Delta\theta_i = -\eta\nabla\ell_i(\theta_0)$, so it points downhill on that panel's loss.

A minibatch averages the selected loss surfaces



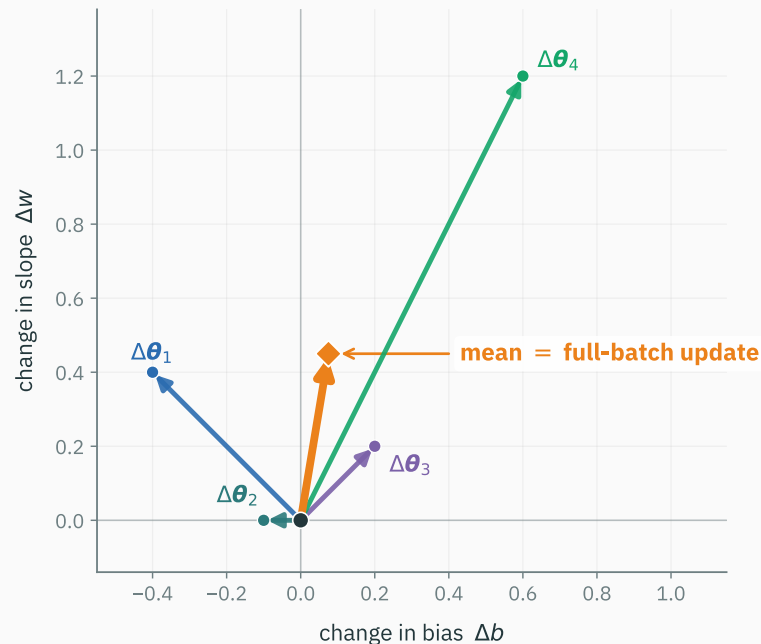
same θ_0 and $\eta = 0.10$ · averaging examples changes both the loss surface and its gradient

COMPUTED FROM EXAMPLE A · ROW 2, ROWS 1-2, AND ALL FOUR ROWS

$$\mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell_i(\theta) \quad \rightarrow \quad \nabla \mathcal{L}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i.$$

Increasing B averages more troughs. Their disagreements partly cancel, and the full batch leaves one exact empirical-loss surface.

Average the one-example updates



COMPUTED FROM EXAMPLE A AT FIXED θ_0

Translate the four updates to a common origin, then average coordinates:

$$\frac{1}{4} \sum_{i=1}^4 \Delta\theta_i = (0.075, 0.450).$$

$$-\frac{\eta}{4} \sum_{i=1}^4 \nabla \ell_i(\theta_0) = \Delta\theta_{\text{full}}.$$

Different samples give different updates; their arithmetic mean is the full-batch update.

An estimator maps random data to an estimate

For a generic discrete random variable X with probability mass $p(x)$, expectation is the probability-weighted average

$$\mathbb{E}[f(X)] = \sum_{x \in \mathcal{X}} p(x) f(x).$$

RANDOM DRAW

$X = x$ occurs with probability $p(x)$

ESTIMATE

apply the rule f and observe $f(X)$

TARGET

a fixed quantity μ to be estimated

The estimator $f(X)$ is **unbiased** for μ when $\mathbb{E}[f(X)] = \mu$.

For continuous X , replace the sum by an integral. Unbiasedness describes the centre across repeated draws—not one draw.

Hold θ_t fixed and substitute into the generic expectation:

$$X \rightarrow I_t, \quad p(i) = \mathbb{P}(I_t = i \mid \theta_t) = \frac{1}{N}, \quad f(i) = \nabla_{\theta} \ell_{i(\theta_t)}.$$

The stochastic gradient estimator is the gradient of the sampled loss:

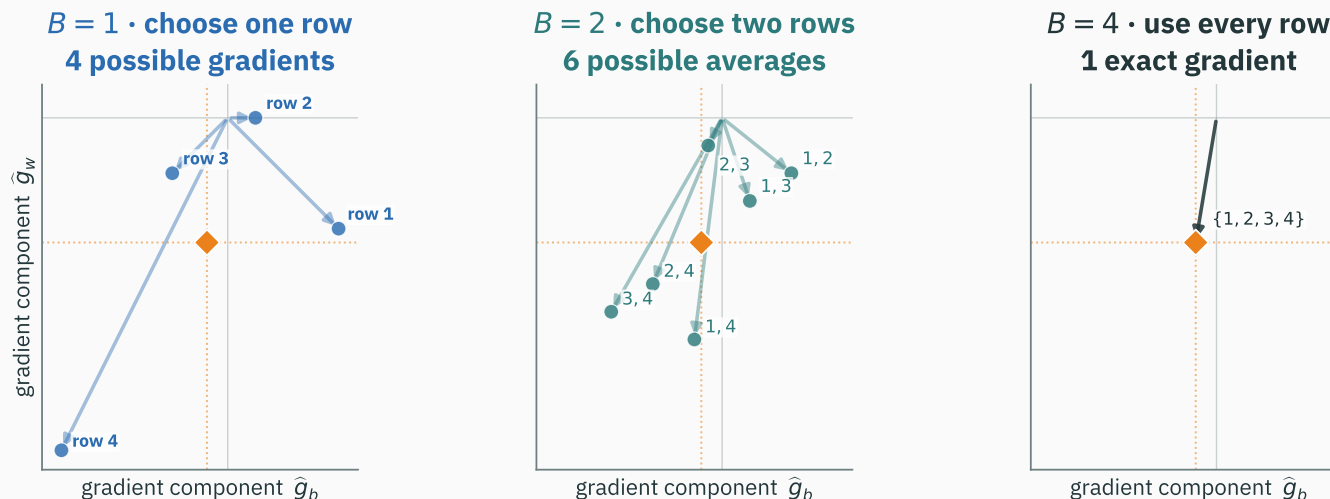
$$\hat{g}_t = f(I_t) = \nabla_{\theta} \ell_{I_t}(\theta_t).$$

$$\mathbb{E}[\hat{g}_t \mid \theta_t] = \sum_{i=1}^N p(i) f(i) = \sum_{i=1}^N \frac{1}{N} \nabla_{\theta} \ell_{i(\theta_t)} = \nabla \left[\frac{1}{N} \sum_{i=1}^N \ell_{i(\theta_t)} \right] = g_t.$$

For Example A, the mean of $(4, -4), (1, 0), (-2, -2), (-6, -12)$ is $(-0.75, -4.5)$ —the full gradient computed earlier.

Minibatches preserve the mean and reduce the spread

A minibatch estimator is simply an **average of one-row estimators**: $\hat{g}_{B,t} = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla \ell_{i(\theta_t)}$.



labels identify the selected row(s) · orange diamond: the same mean = g_{full} in every panel

EXACT ENUMERATION OF EVERY UNIFORM EXAMPLE A SUBSET AT THE SAME θ_0

Same centre: $\mathbb{E}[\hat{g}_{B,t} \mid \theta_t] = g_t$. **Less spread:** larger B cancels more disagreement.

Unbiased means correct on average—not downhill every time

Our earlier valid SGD draw sampled row 2 and used $\hat{g}_0 = (1, 0)$, so $\theta_1 = (1.9, -1)$.

SAMPLED LOSS

$$\ell_2 : 0.500 \rightarrow 0.405$$

The update succeeds on the sampled row.

FULL LOSS

$$\mathcal{L} : 7.125 \rightarrow 7.205$$

The same update is uphill for the average loss.

Unbiasedness guarantees $\mathbb{E}[\hat{g}_t \mid \theta_t] = g_t$ —equivalently, $\mathbb{E}[\Delta\theta_t \mid \theta_t] = -\eta g_t$. It does not make every estimate close, every step downhill, or convergence automatic.

1 • INSPECT

Plot each ℓ_i , a minibatch average, and the full $\mathcal{L}$.

2 • VERIFY

Enumerate the possible gradients and check their mean.

3 • EXPERIMENT

Change B and watch the estimator cloud contract.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L05/00_full_batch_sgd_minibatch.ipynb

Open **Full batch, SGD, minibatches, and gradient estimators** in Colab ↗

**An exact gradient can still be
slow**

Example B: one feature is ten times larger

Use $\hat{y}_i = \theta_1 x_{i1} + \theta_2 x_{i2}$ and generate targets with $\theta^* = (1, -0.5)$.

The first feature is ± 1 ; the second is ± 10 . This deliberate scale difference creates a transparent optimization problem.

i	x_{i1}	x_{i2}	y_i
1	-1	-10	4
2	-1	10	-6
3	1	-10	6
4	1	10	-4

We will stay in the original parameter space throughout:

$$\theta_0 = (-1.4, 0.35).$$

The optimum is $\theta^* = (1, -0.5)$. We will plot and update θ directly—no second coordinate system is needed.

The $10 \times$ feature creates a $100 \times$ sensitive direction

For these four sign combinations, the mixed terms cancel when we average the half-squared residuals:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}(\theta_1 - 1)^2 + 50(\theta_2 + 0.5)^2, \quad \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = (\theta_1 - 1, 100(\theta_2 + 0.5)).$$

MOVE θ_1 BY THE SAME AMOUNT

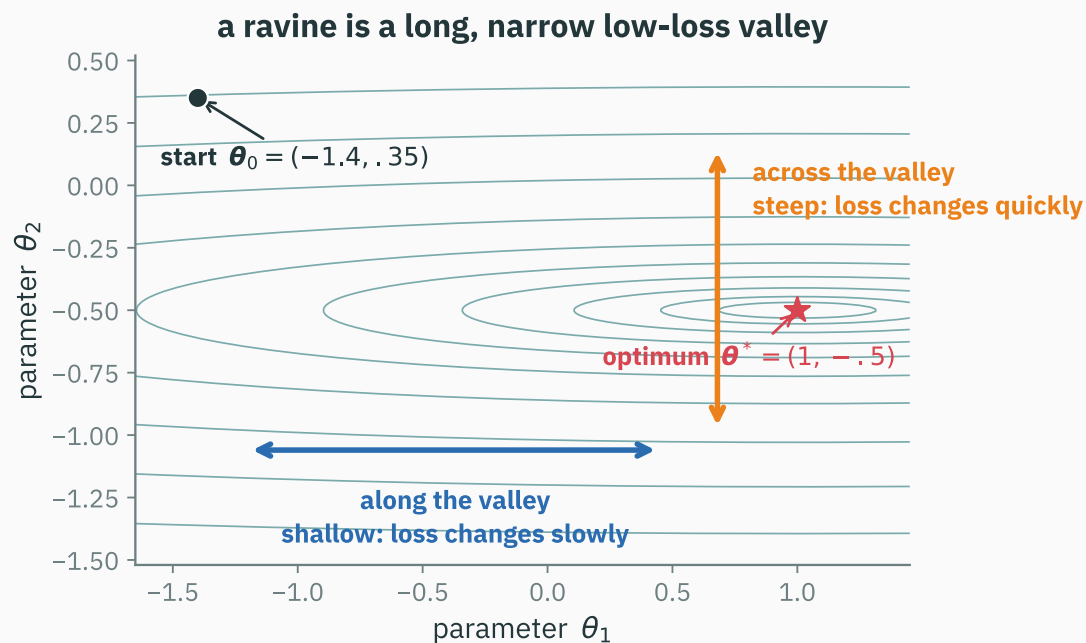
$$\text{loss contribution} = \frac{1}{2}(\theta_1 - 1)^2$$

MOVE θ_2 BY THE SAME AMOUNT

loss contribution = $50(\theta_2 + 0.5)^2$: one hundred times larger

Curvature means how quickly the slope changes as we move. The second coordinate is much more sensitive because the feature scale was squared.

A ravine is a long, narrow low-loss valley



computed from the four displayed rows · each contour joins points with the same full-batch loss

COMPUTED FROM THE FOUR DISPLAYED REGRESSION EXAMPLES · EXACT FULL-BATCH LOSS

One correct step can overshoot across the valley

At $\theta_0 = (-1.4, .35)$,

$g_0 = (-2.4, 85)$.

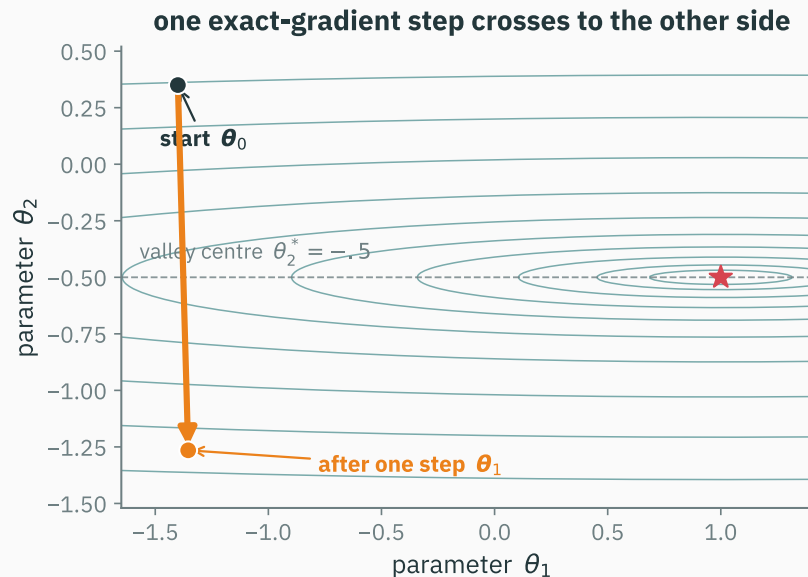
With $\eta = .019$,

$\Delta\theta_0 = (.0456, -1.615)$.

Therefore

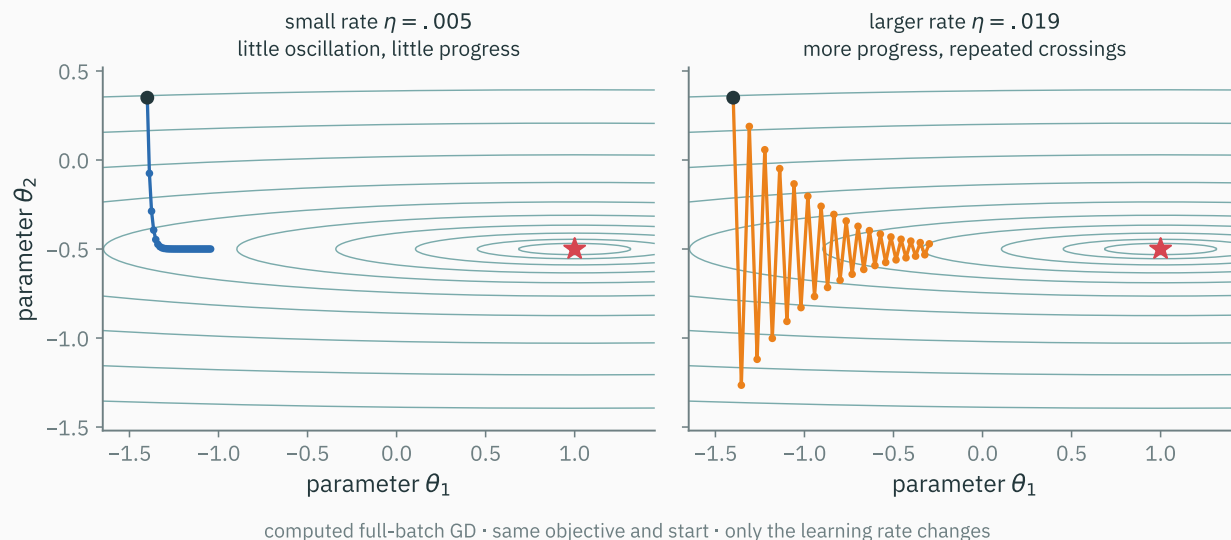
$\theta_1 = (-1.3544, -1.265)$.

g_0 points uphill; $\Delta\theta_0 = -\eta g_0$
points downhill. This step
overshoots $\theta_2^* = -.5$.



COMPUTED EXACT-GRADIENT UPDATE

The trade-off is visible in the computed paths



computed full-batch GD · same objective and start · only the learning rate changes

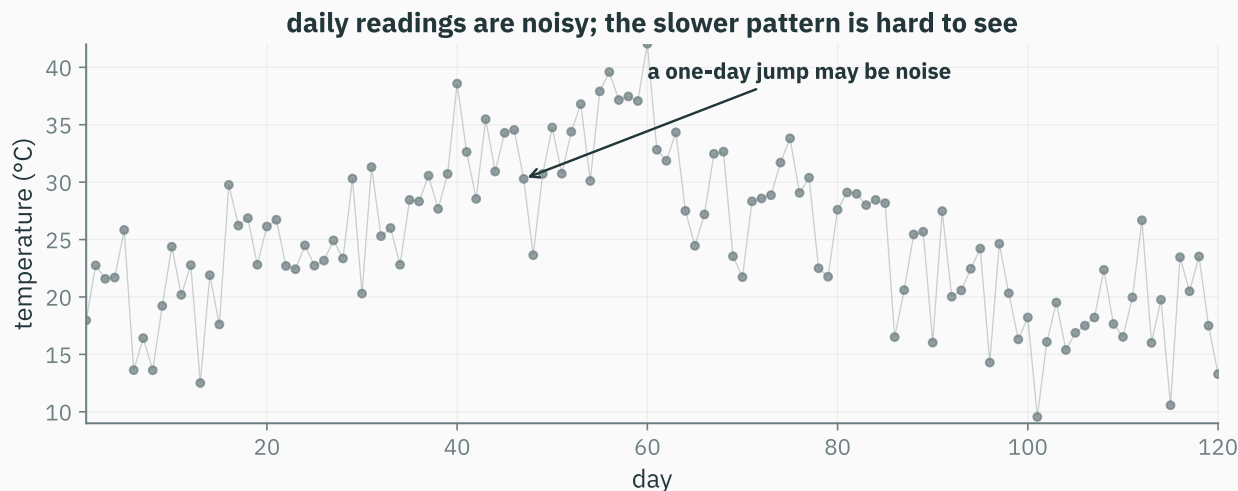
COMPUTED FULL-BATCH GD · SAME DATA, START, AND UPDATE BUDGET

The small rate avoids crossing $\theta_2^* = -0.5$, but barely changes θ_1 . The larger rate moves θ_1 faster and repeatedly crosses the optimum in θ_2 . One global learning rate creates this trade-off.

**How can we smooth a noisy
sequence?**

Start with the noisy observations—not the formula

I INTERACTIVE

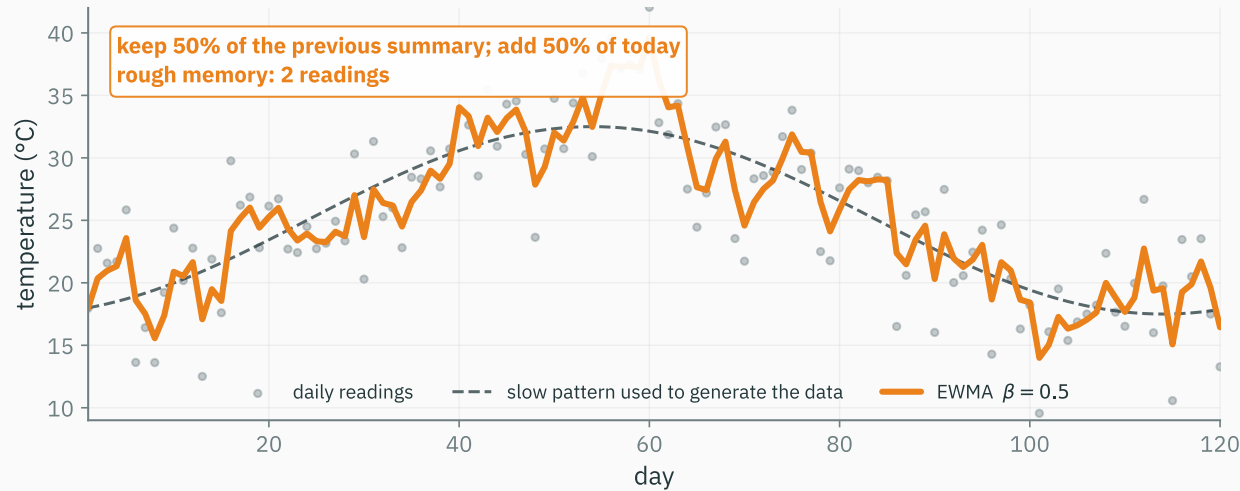


CONSTRUCTED 120-DAY TEMPERATURE SIGNAL • FIXED SEED 12

We want a running estimate of the changing trend. A **memory** is one summary carried from yesterday to today; it does not store every reading.

Open the [EWMA + Momentum Explorer ↗](#) — tune memory, then step through the actual ravine.

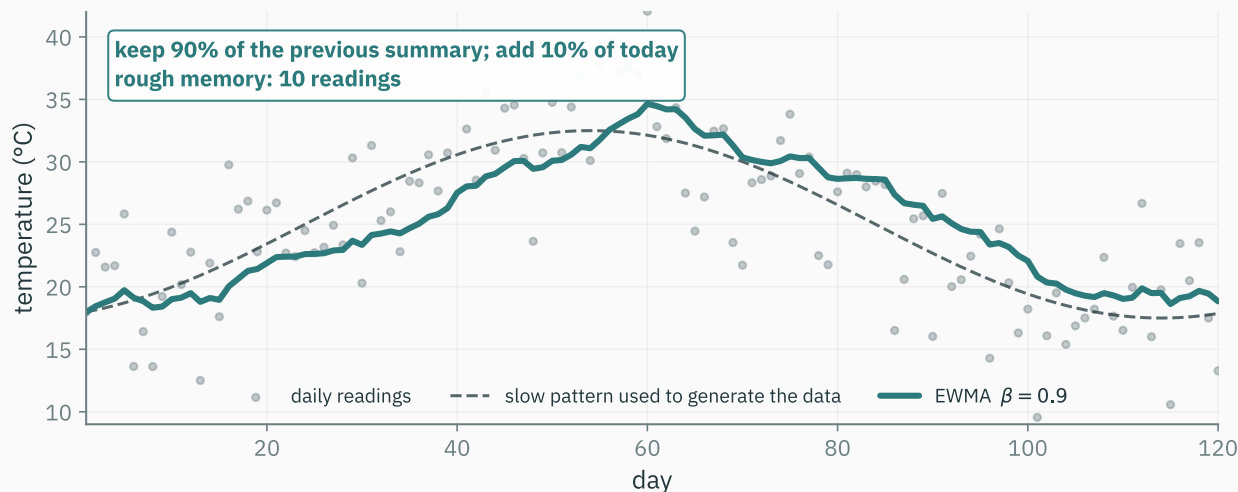
Short memory reacts quickly—and follows the noise



SAME READINGS • EWMA INITIALIZED AT THE FIRST READING

β is the fraction of the previous summary retained. At $\beta = .5$, the previous summary and today's reading receive equal weight, so the curve remains jumpy.

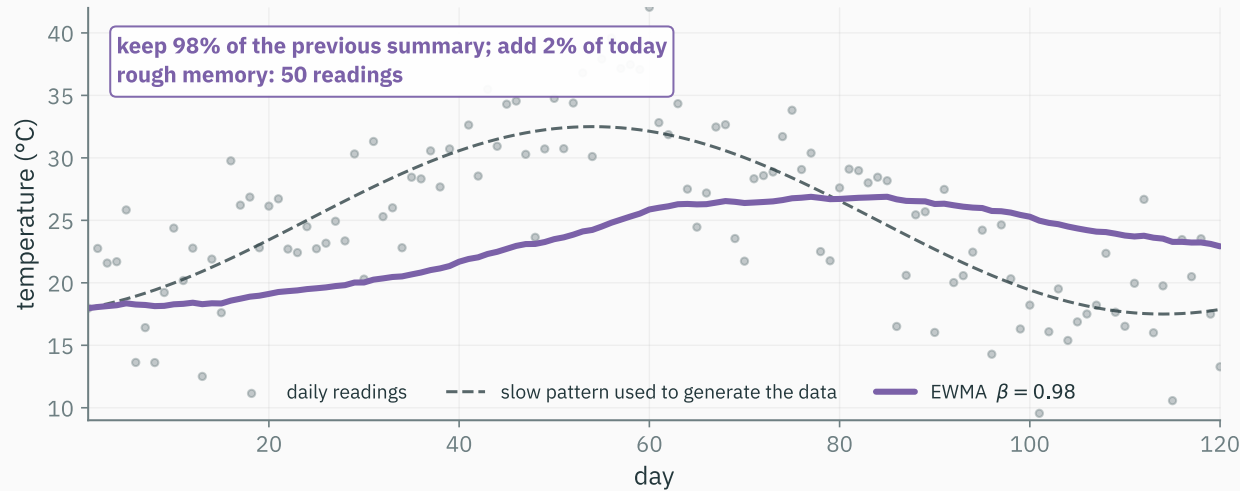
More memory gives a useful compromise



SAME READINGS · ONLY β CHANGES

At $\beta = .9$, keep 90% of the previous summary and add 10% of today's reading.
Noise is reduced, with some delay when the trend turns.

Too much memory becomes slow to respond



SAME READINGS · ONLY β CHANGES

$\beta = .98$ averages over a much longer effective history. It is very smooth, but it visibly lags the changing signal.

$$\underbrace{m_t}_{\text{running memory after today's reading}} = \underbrace{\beta m_{t-1}}_{\text{keep the previous summary}} + \underbrace{(1 - \beta)x_t}_{\text{add today's observation}}, \quad 0 \leq \beta < 1.$$

ONE WORKED UPDATE

$$m_{t-1} = 30, x_t = 36, \beta = .9$$

$$m_t = .9(30) + .1(36) = 30.6$$

WHAT IS CARRIED FORWARD?

Keep the new memory $m_t = 30.6$ for the next observation. The earlier readings need not be stored.

This running summary is an **exponentially weighted moving average (EWMA). For the temperature curves we set $m_1 = x_1$; optimizer momentum below instead starts from $m_0 = 0$.**

Why “exponential”? Unroll the memory

Substitute the same recurrence for the previous memory m_{t-1} :

$$m_t = \beta[\beta m_{t-2} + (1 - \beta)x_{t-1}] + (1 - \beta)x_t$$

$$m_t = (1 - \beta)x_t + (1 - \beta)\beta x_{t-1} + \beta^2 m_{t-2}.$$

Substitute once more and the pattern appears:

$$m_t = (1 - \beta)x_t + (1 - \beta)\beta x_{t-1} + (1 - \beta)\beta^2 x_{t-2} + \beta^3 m_{t-3}.$$

GENERAL PATTERN

The weight on x_{t-k} is $(1 - \beta)\beta^k$. Every step into the past contributes one more factor of β —this geometric decay is the “exponential” in EWMA.

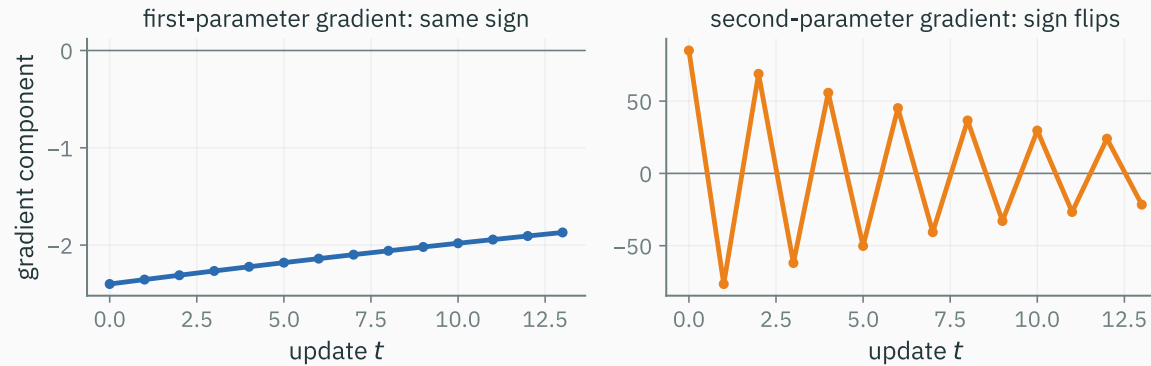
$$\beta = .9$$

Recent-observation weights are:

$$x_t \rightarrow .100 \quad x_{t-1} \rightarrow .090$$

$$x_{t-2} \rightarrow .081 \quad x_{t-3} \rightarrow .0729 \quad \dots$$

Same-direction pushes build speed; reversals cancel



full-batch gradients along the actual $\eta = .019$ path $\cdot \mathbf{g}_t = (\theta_{t,1} - 1, 100(\theta_{t,2} + .5))$

COMPUTED • EXAMPLE B • ACTUAL FULL-BATCH PATH • NOTEBOOK 02

Plain GD uses only the current push. In θ_1 , successive gradients keep the same sign; in θ_2 , they reverse. A running memory therefore reinforces motion in θ_1 while cancelling much of the zigzag in θ_2 . Think rolling ball—not literal physics.

Momentum update 1: there is no history yet

Momentum stores a running memory m_t of earlier gradients. Before update 1, it has seen none, so we choose the initial condition

$$m_0 = 0.$$

Zero is not a measured gradient or a learned parameter. It means **no history and no preferred direction**. Hence $m_1 = (1 - \beta)g_0$; earlier gradients enter only from update 2.

For this example, $\beta = .8$ and $\eta = .095$: keep 80% of the old memory and add 20% of the current full-batch gradient.

1 • MEASURE

At θ_0 ,

$$g_0 = (-2.4, 85)$$

2 • REMEMBER

$$\begin{aligned} m_1 &= .8m_0 + .2g_0 \\ &= .2g_0 = (-.48, 17) \end{aligned}$$

3 • MOVE

$$\begin{aligned} \theta_1 &= \theta_0 - .095m_1 \\ \theta_1 &= (-1.3544, -1.265) \end{aligned}$$

At the new point θ_1 , keep $m_1 = (-.48, 17)$ and **remeasure**:

$$g_1 = (-2.3544, -76.5), \quad m_2 = .8m_1 + .2g_1.$$

FIRST PARAMETER: SAME SIGN

$$m_{2,1} = .8(-.48) + .2(-2.3544)$$

$$= -.384 - .4709 = -.8549$$

Same sign $\rightarrow$ memory grows.

SECOND PARAMETER: SIGN REVERSES

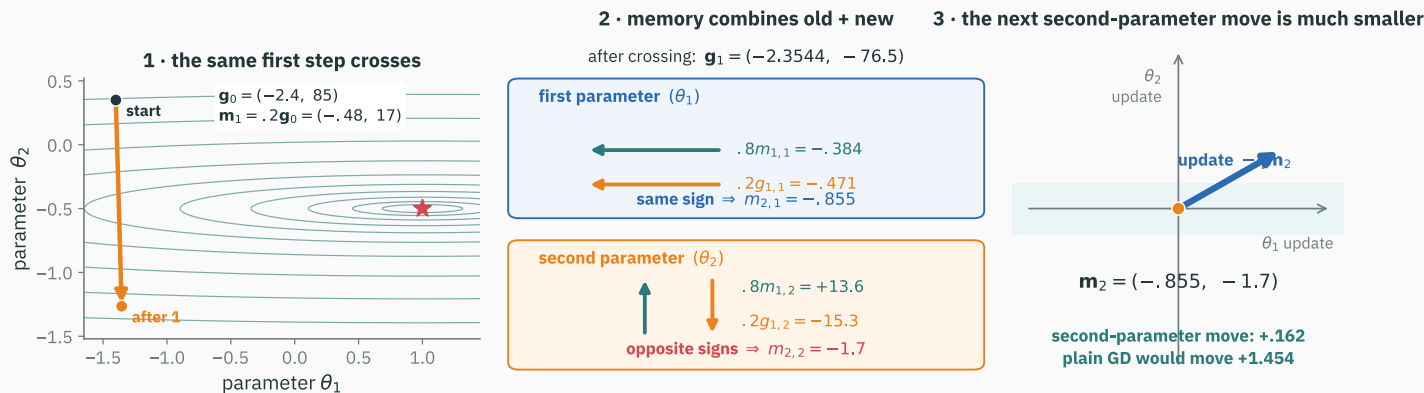
$$m_{2,2} = .8(17) + .2(-76.5)$$

$$= 13.6 - 15.3 = -1.7$$

Sign reversal $\rightarrow$ memory nearly cancels.

$$\textbf{Move: } \theta_2 = \theta_1 - .095m_2 = (-1.2732, -1.1035).$$

What changed in the second momentum step?

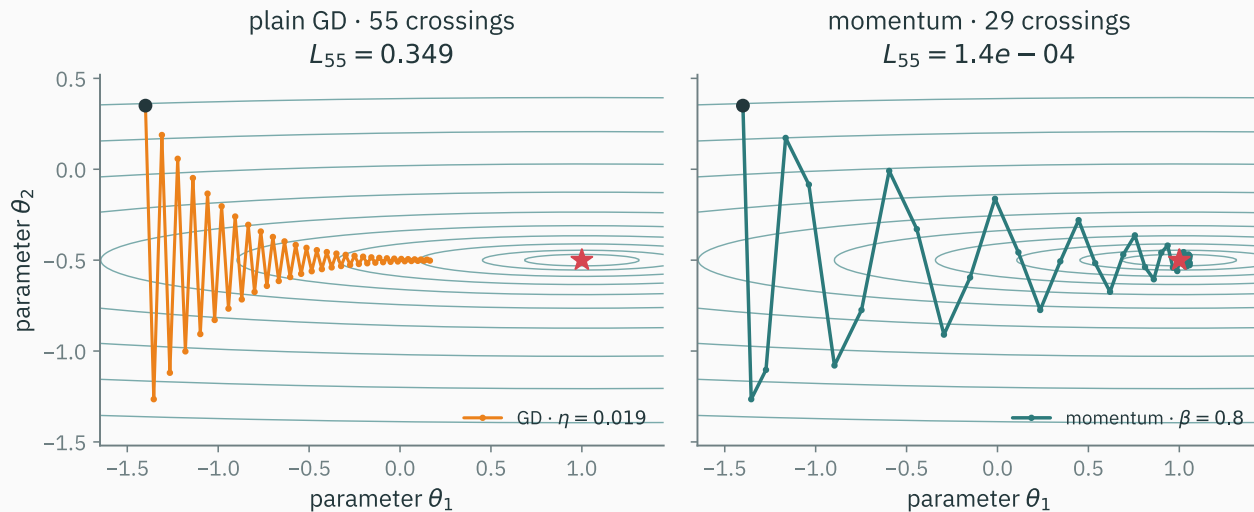


computed from Example B · first displacement matches GD $\eta = .019$ · normalized memory $\beta = .8, \eta = .095$

COMPUTED FROM THE TWO WORKED UPDATES · EXAMPLE B

The first-parameter contributions agree, so memory strengthens that motion. The second-parameter contributions disagree, so memory cancels most of the reversal before the update is applied.

Momentum travels farther in the same 55 updates

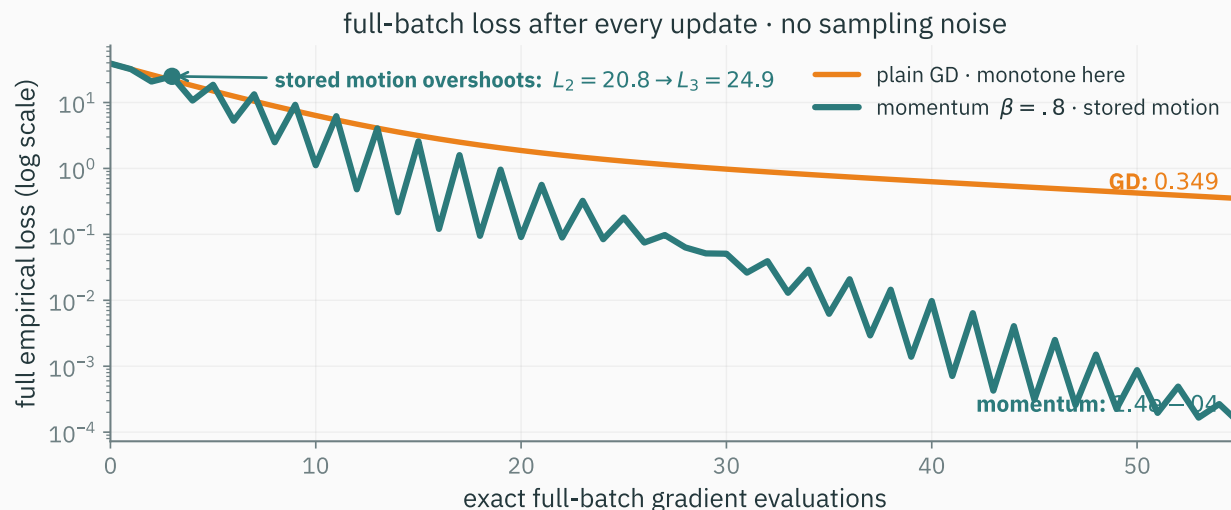


55 exact full-batch gradient evaluations each · same start and first displacement · GD $\eta = .019$ · normalized momentum $\eta = .095$, $\beta = .8$

COMPUTED · SAME START AND FIRST STEP · 55 EXACT FULL-BATCH GRADIENT EVALUATIONS EACH

Both methods start from the same θ_0 and make the same first move. Momentum makes more progress in θ_1 while reducing the size of the alternating θ_2 moves. The full-loss slide next checks whether that helps.

Momentum is faster here—not downhill at every update



COMPUTED ON EXAMPLE B · IDENTICAL GRADIENT-EVALUATION BUDGET

Both curves use exact full-batch gradients. At this stable rate, plain GD shrinks both errors from the optimum, $|\theta_1 - 1|$ and $|\theta_2 + 0.5|$, every update, so its loss falls monotonically. Momentum carries stored motion past the valley: $\mathcal{L}_2 = 20.8 \rightarrow \mathcal{L}_3 = 24.9$. Temporary uphill steps are allowed; compare the overall trend and final loss.

Under the hood: one memory tensor per parameter

I INTERACTIVE

```
beta, eta = 0.8, 0.095
m = [torch.zeros_like(p) for p in params]

for xb, yb in loader:
    loss = batch_loss(params, xb, yb)
    grads = torch.autograd.grad(loss, params)      # measure
    with torch.no_grad():
        for p, g, state in zip(params, grads, m):
            state.mul_(beta).add_(g, alpha=1-beta) # remember
            p.add_(state, alpha=-eta)              # move
```

The current gradient is recomputed at every update. Only the parameter-shaped memory persists from one update to the next.

PyTorch momentum uses the same training loop

I INTERACTIVE

```
opt = torch.optim.SGD(model.parameters(),  
                        lr=0.019, momentum=0.8)  
  
opt.zero_grad(set_to_none=True)  
loss.backward()  
opt.step()
```

SIGNAL

Compare $\beta =$
.5, .9, .98.

OPTIMIZER

Reproduce the
two worked
updates.

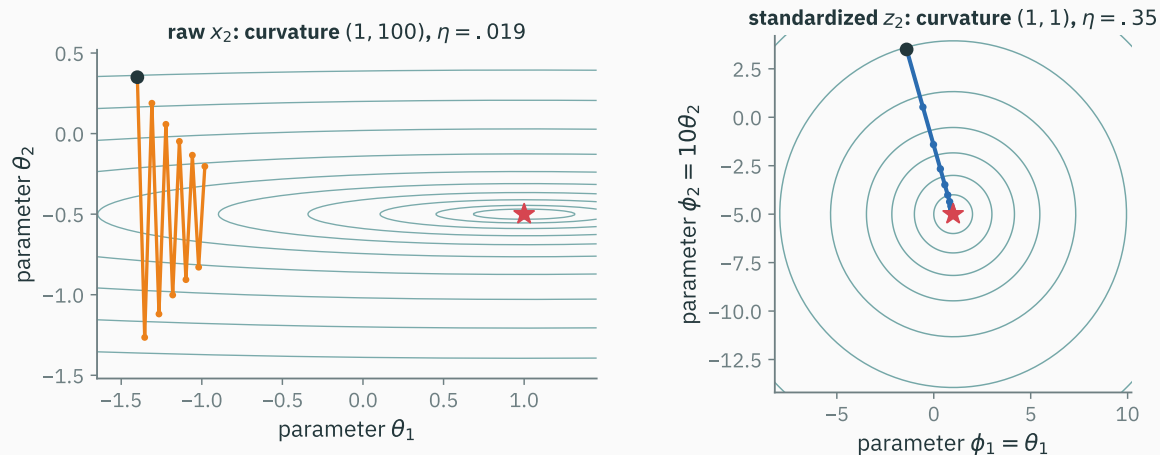
PyTorch stores an unnormalized **momentum buffer**: $\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \mathbf{g}_t$ and $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \text{lr}_{\text{PT}} \mathbf{v}_{t+1}$. Since $\mathbf{m}_{t+1} = (1 - \beta) \mathbf{v}_{t+1}$, the two forms match when $\text{lr}_{\text{PT}} = (1 - \beta) \eta = .019$.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L05/02_ewma_momentum.ipynb

Open **Ravines**, **EWMA**, and **momentum** in Colab ↗

Could feature standardization fix this example? Yes.



10 updates each · same starting predictor: $\theta_0 = (-1.4, .35)$ maps to $\phi_0 = (-1.4, 3.5)$

COMPUTED BEFORE/AFTER GEOMETRY FOR $z_2 = \frac{x_2}{10}$

Here standardizing x_2 removes the 100 : 1 scale imbalance. With a stable larger rate, plain GD moves directly toward the optimum—momentum is not needed for this constructed problem.

Feature scaling helps; it does not make momentum obsolete

DO FIRST

Normalize input features when their scales differ strongly.

WHAT REMAINS

Deep networks keep changing how parameters interact. Input scaling cannot make every training direction equally easy.

AND MINIBATCHES

Sampled gradients still fluctuate even with well-scaled inputs.

Normalization changes the geometry. Momentum changes how successive gradients are combined. They are complementary.

For the next controlled experiment, we deliberately return to the original unscaled ravine so the coordinate-scale problem remains visible.

**One learning rate cannot fit
both directions**

The problem: one coordinate moves 35× farther

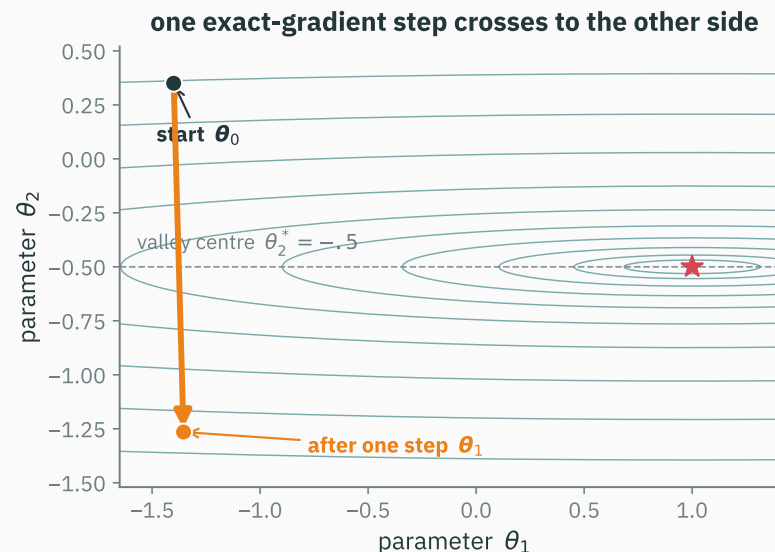
At the same start, $g_0 = (-2.4, 85)$.

Plain GD uses the same η in both coordinates, so

$$\left| \frac{g_{0,2}}{g_{0,1}} \right| = 35.4.$$

A larger global rate helps along the valley but worsens the across-valley step.

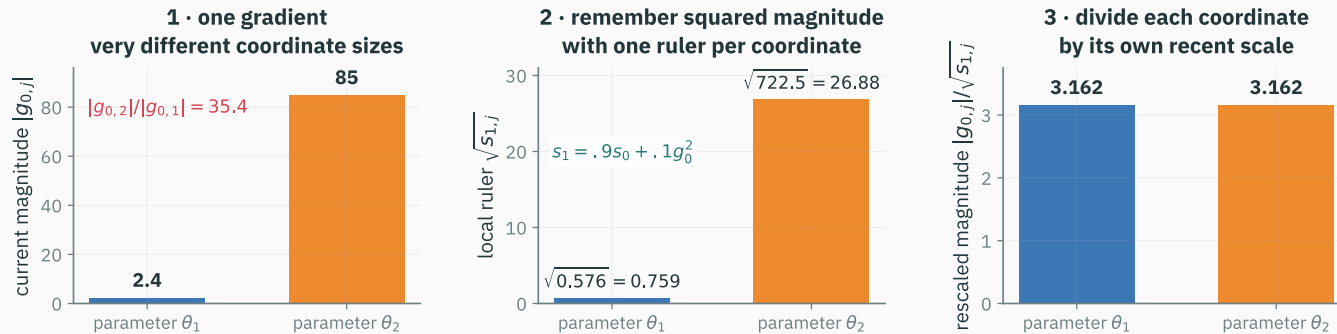
RMSProp will learn a separate scale for each coordinate.



COMPUTED FROM EXAMPLE B AT $\eta = .019$

RMSProp intuition: give each coordinate its own ruler

Learn a recent magnitude for each coordinate, then measure today's gradient against its own ruler rather than one shared scale.



Equal first magnitudes are a zero-start effect: later gradients update each ruler, so the scaled coordinates need not remain equal.

computed from Example B at $\theta_0 = (-1.4, .35) \cdot \rho = .9 \cdot \varepsilon$ omitted only from this arithmetic illustration

COMPUTED FROM $g_0 = (-2.4, 85) \cdot \rho = .9 \cdot \text{ZERO-START FIRST STATE}$

Squaring removes sign; the square root restores gradient units. A coordinate with a large recent ruler is divided more strongly.

The rule: remember squared magnitude, then rescale

D DERIVATION

MEASURE

$\hat{g}_t$: current full-batch or minibatch gradient

REMEMBER SCALE

EWMA of $\hat{g}_t^2$ in s_{t+1}

MOVE

divide each coordinate by its learned ruler

The rule: remember squared magnitude, then rescale

D DERIVATION

MEASURE

$\hat{g}_t$: current full-batch or minibatch gradient

REMEMBER SCALE

EWMA of $\hat{g}_t^2$ in s_{t+1}

MOVE

divide each coordinate by its learned ruler

$$s_{t+1} = \rho s_t + (1 - \rho) \hat{g}_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{g}_t}{\sqrt{s_{t+1}} + \varepsilon_{\text{opt}}}.$$

| Every square, root, and division is elementwise. ε_{opt} keeps a nearly zero ruler safe.

Work one RMSProp update from start to finish

D DERIVATION

Use $g_0 = (-2.4, 85)$, $\rho = .9$, and $s_0 = 0$. To keep this one hand calculation readable, omit ε_{opt} here; the implementation retains it.

1 • SQUARE + REMEMBER

$$\begin{aligned}s_1 &= .1g_0^2 \\ &= (.576, 722.5)\end{aligned}$$

2 • BUILD TWO RULERS

$$\begin{aligned}\sqrt{s_1} \\ &= (.759, 26.879)\end{aligned}$$

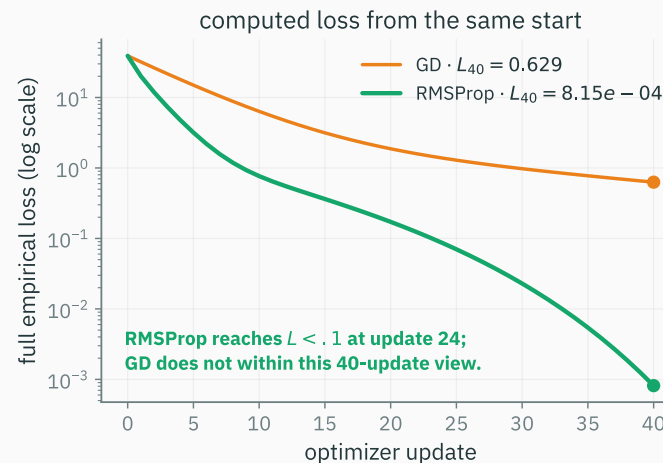
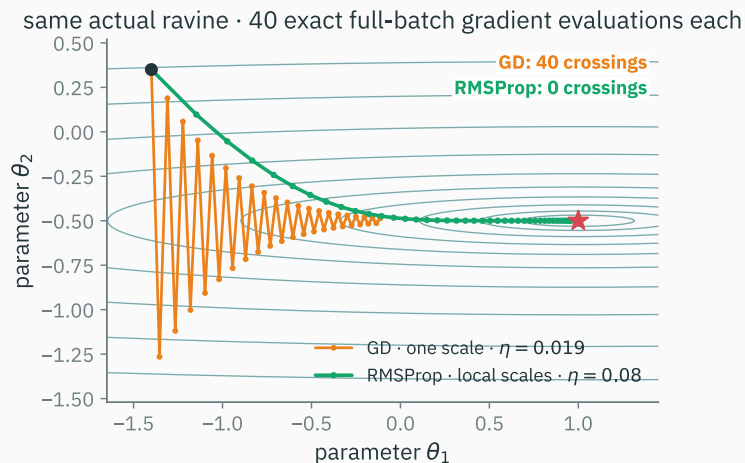
3 • RESCALE

$$\begin{aligned}\frac{g_0}{\sqrt{s_1}} \\ &= (-3.162, 3.162)\end{aligned}$$

With $\eta = .08$ **and** $\theta_0 = (-1.4, .35)$, $\theta_1 = \theta_0 - .08(-3.162, 3.162) \approx (-1.147, .097)$.

The equal first magnitudes are a **zero-start effect**, not a permanent promise. Later rulers contain the actual gradient history; Notebook 03 traces the second state.

Computed outcome: the ravine path straightens



computed Example B · 40 exact full-batch gradients · GD $\eta = 0.019$ · RMSProp $\eta = 0.08$, $\rho = 0.9$, $\varepsilon = 1e-08$ · method-specific rates, not a benchmark

COMPUTED · SAME OBJECTIVE · 40 EXACT FULL-BATCH GRADIENT EVALUATIONS EACH · METHOD-SPECIFIC RATES SHOWN

RMSProp now makes progress along the shallow direction without reproducing GD's large repeated crossings. This is a controlled mechanism example, not a universal ranking.

```
opt = torch.optim.RMSprop(model.parameters(), lr=0.08,  
                           alpha=0.9, eps=1e-8)  
  
opt.zero_grad(set_to_none=True)  
loss.backward()  
opt.step()  
  
state = opt.state[next(model.parameters())]  
print(state["square_avg"]) # the tensor s_(t+1)
```

PyTorch computes $\sqrt{s_{t+1}} + \varepsilon_{\text{opt}}$. The notebook checks both square_avg and the parameter update after every displayed step.

**Adam: remember direction and
scale**

Why combine them? the failures are different

MOMENTUM REMEMBERS DIRECTION

Persistent signs accumulate; alternating signs partly cancel. It does not learn a separate coordinate scale.

RMSPROP REMEMBERS SCALE

Large recent magnitudes build a large ruler. It does not remember a signed direction.

Why combine them? the failures are different

MOMENTUM REMEMBERS DIRECTION

Persistent signs accumulate; alternating signs partly cancel. It does not learn a separate coordinate scale.

RMSPROP REMEMBERS SCALE

Large recent magnitudes build a large ruler. It does not remember a signed direction.

Adam asks both questions: which direction persists? and how large is this coordinate usually?

Direction memory goes in the numerator; scale memory goes in the denominator.

Adam runs two memories in parallel

D DERIVATION

m_t · direction memory EWMA of the signed
gradient

v_t · scale memory EWMA of the squared
gradient

Adam runs two memories in parallel

D DERIVATION

$\mathbf{m}_t$ · direction memory EWMA of the signed gradient

$\mathbf{v}_t$ · scale memory EWMA of the squared gradient

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \hat{\mathbf{g}}_t$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \hat{\mathbf{g}}_t^2$$

$$\Delta \theta_t = -\eta \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon_{\text{opt}}}}$$

Two parameter-shaped states plus a step counter are carried across updates. The learning rate still matters.

Zero-start correction makes early memories comparable

Both memories start at zero. Early in training, their exponential weights have not yet accumulated to one, so the raw memories are artificially small.

CORRECT DIRECTION MEMORY

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}$$

CORRECT SCALE MEMORY

$$\hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$$

Zero-start correction makes early memories comparable

Both memories start at zero. Early in training, their exponential weights have not yet accumulated to one, so the raw memories are artificially small.

CORRECT DIRECTION MEMORY

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}$$

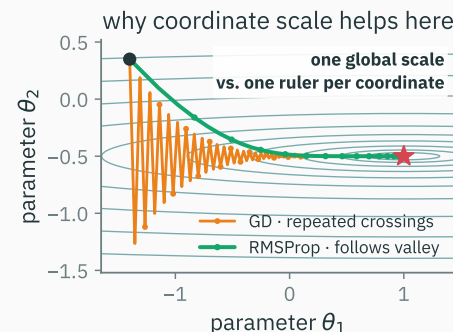
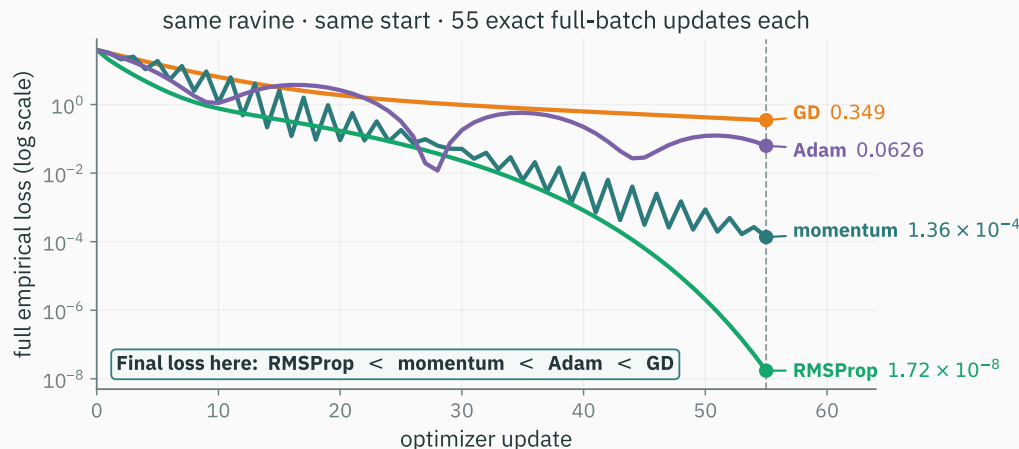
CORRECT SCALE MEMORY

$$\hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$$

At the first update, $\hat{m}_1 = \hat{g}_0$ and $\hat{v}_1 = \hat{g}_0^2$: correction removes the artificial zero-start shrinkage.

For Example B, with $\eta = .10$ and ε_{opt} omitted only for this arithmetic, $\Delta\theta_0 = -.10(-1, 1) = (.10, -.10)$, so $\theta_1 = (-1.30, .25)$.

Same ravine, four optimizers



method-specific settings: GD $\eta = .019$ · normalized momentum $\eta = .095, \beta = .8$ (PyTorch lr = .019) · RMSProp $\eta = .08, \rho = .9$ · Adam $\eta = .10, \beta = (.9, .999)$
controlled Example B mechanism comparison · method-specific rates · not a universal optimizer ranking

COMPUTED · SAME FULL-BATCH OBJECTIVE · SAME 55-UPDATE BUDGET · ALL METHOD-SPECIFIC RATES SHOWN

On this example after 55 updates: **RMSProp < momentum < Adam < GD**.
The plotted values use chosen, method-specific rates—not a universal ranking.

PyTorch exposes `exp_avg` and `exp_avg_sq`

```
opt = torch.optim.Adam(model.parameters(), lr=.10,  
                        betas=(0.9, 0.999), eps=1e-8)  
  
opt.zero_grad()  
loss.backward()  
opt.step()  
  
state = opt.state[next(model.parameters())]  
print(state.keys()) # step, exp_avg, exp_avg_sq
```

`exp_avg` is m_{t+1} ; `exp_avg_sq` is v_{t+1} . These are the two memories derived on the previous slides.

Choose state to match the failure

PLAIN GD / SGD · NO STATE

Start here: cheap and often strong. It does not address sampling noise or unequal curvature by itself.

MOMENTUM · m

Use when gradient directions persist or alternate. It does not learn coordinate rulers.

RMSPROP · s

Use when recent coordinate magnitudes differ. It does not remember signed direction.

ADAM · m, v

Use when both effects matter. It still cannot repair bad data, a wrong loss, or poor tuning.

MECHANISM

Predict the two rulers, trace both Adam memories, then run each method on the exact ravine.

FRAMEWORK CHECK

Compare every state and parameter update to `torch.optim.RMSprop` and `torch.optim.Adam`.

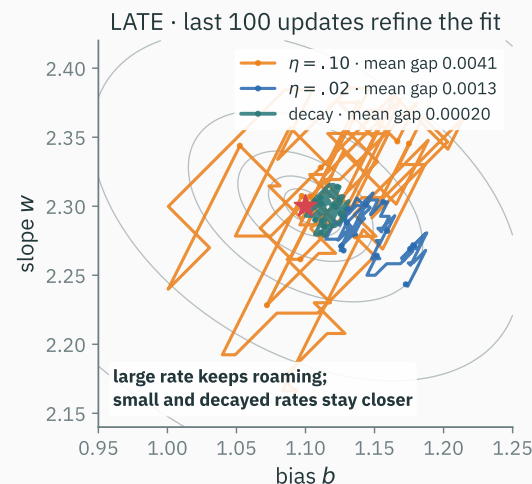
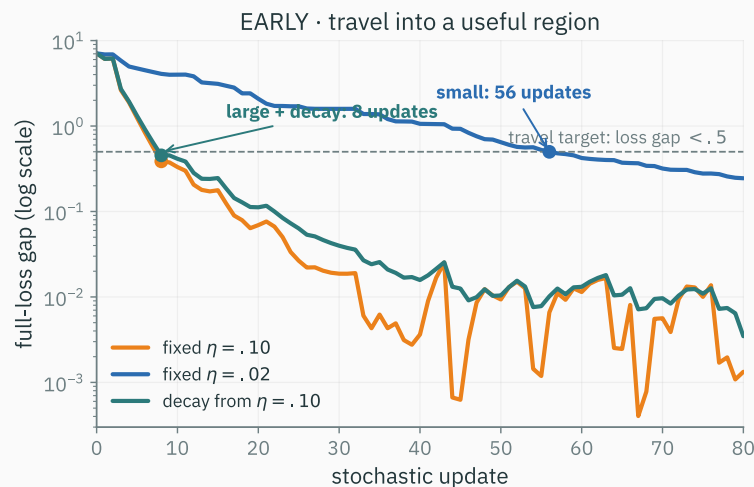
INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L05/03_adaptive_optimizers_constraints.ipynb

Open **Adaptive optimizers, schedules, and constraints** in Colab ↗

**Let the step size change with
training**

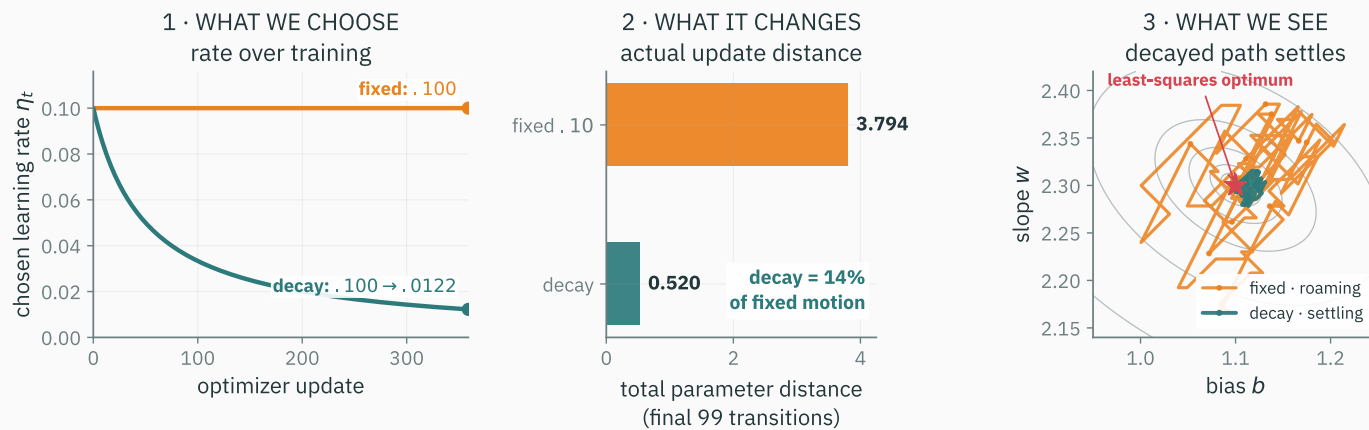
One schedule has two jobs: travel early, refine late



COMPUTED · EXAMPLE A · IDENTICAL SAMPLED-INDEX STREAM · SEED 12

A large fixed rate travels quickly but keeps wandering. A small fixed rate settles more tightly but wastes early updates. Decay starts large and finishes small.

Decay shrinks late updates, so the path settles



COMPUTED · EXAMPLE A · IDENTICAL SAMPLED-INDEX STREAM · SEED 12

Same start, optimizer, and sampled-index stream. The fixed-rate run travels $7.3 \times$ as far as the decayed run over the final 99 transitions here.

One simple schedule is enough to understand the idea

$$\eta_t = \frac{\eta_0}{1 + dt}, \quad d > 0.$$

One simple schedule is enough to understand the idea

$$\eta_t = \frac{\eta_0}{1 + dt}, \quad d > 0.$$

clock t	rate	interpretation
0	η_0	fast initial movement
50	$\frac{\eta_0}{1+50d}$	smaller updates
$t \rightarrow \infty$	$\eta_t \rightarrow 0$	late refinement

One simple schedule is enough to understand the idea

$$\eta_t = \frac{\eta_0}{1 + dt}, \quad d > 0.$$

clock t	rate	interpretation
0	η_0	fast initial movement
50	$\frac{\eta_0}{1+50d}$	smaller updates
$t \rightarrow \infty$	$\eta_t \rightarrow 0$	late refinement

Step, exponential, cosine, and warmup schedules change the shape—not the core idea. Always state whether t counts optimizer updates or epochs.

PyTorch: implement the **same** update-clock schedule

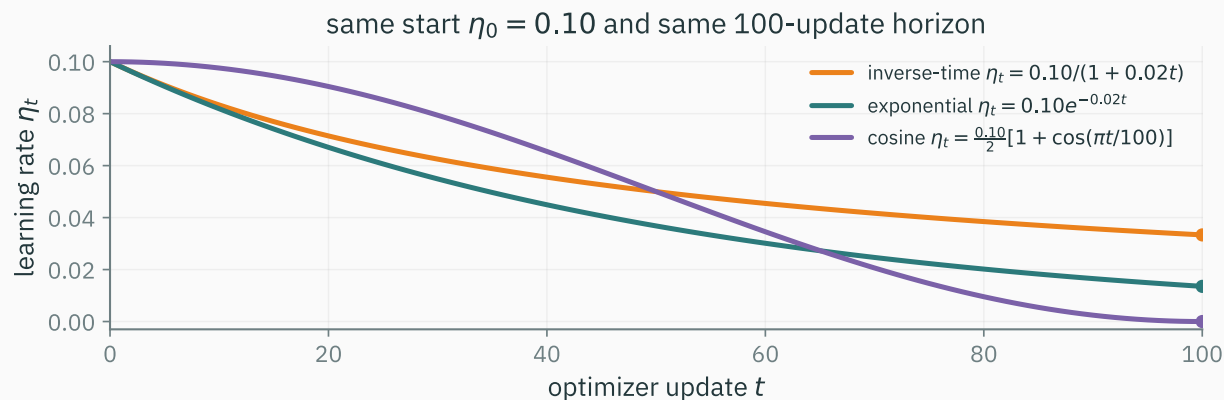
I INTERACTIVE

```
opt = torch.optim.SGD(model.parameters(), lr=.10)
d = .02
sched = torch.optim.lr_scheduler.LambdaLR(
    opt, lr_lambda=lambda t: 1.0 / (1.0 + d*t))

for xb, yb in loader:
    opt.zero_grad(set_to_none=True)
    batch_loss(model, xb, yb).backward()
    opt.step()      # uses eta_t
    sched.step()    # prepares eta_(t+1)
```

The formula, computed figure, notebook, and code now all use inverse-time decay per optimizer update. With gradient accumulation, advance this clock only after `opt.step()`.

Different schedules change the shape, not the clock



schedule shapes only · curves do not imply optimizer performance · inverse-time is applied once per optimizer update

COMPUTED · SAME INITIAL RATE AND UPDATE HORIZON · SCHEDULE SHAPES ONLY

Inverse-time, exponential, and cosine are different choices for η_t . Whatever the shape, state whether t counts optimizer updates, epochs, or another explicit event.

Optimizers on the XOR network

Example C: return to Lecture 2's XOR task

SAME DATA

$(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$,
 $(1, 1) \rightarrow 0$

→

SAME NETWORK

$2 \rightarrow 4 \text{ tanh units} \rightarrow 1 \text{ logit}$

Example C: return to Lecture 2's XOR task

SAME DATA

$(0, 0) \rightarrow 0, (0, 1) \rightarrow 1, (1, 0) \rightarrow 1,$
 $(1, 1) \rightarrow 0$

→

SAME NETWORK

$2 \rightarrow 4 \text{ tanh units} \rightarrow 1 \text{ logit}$

The representation and loss are fixed. We change only the optimizer while keeping initialization, minibatch order, and update budget identical.

A controlled mechanism demo—not a leaderboard

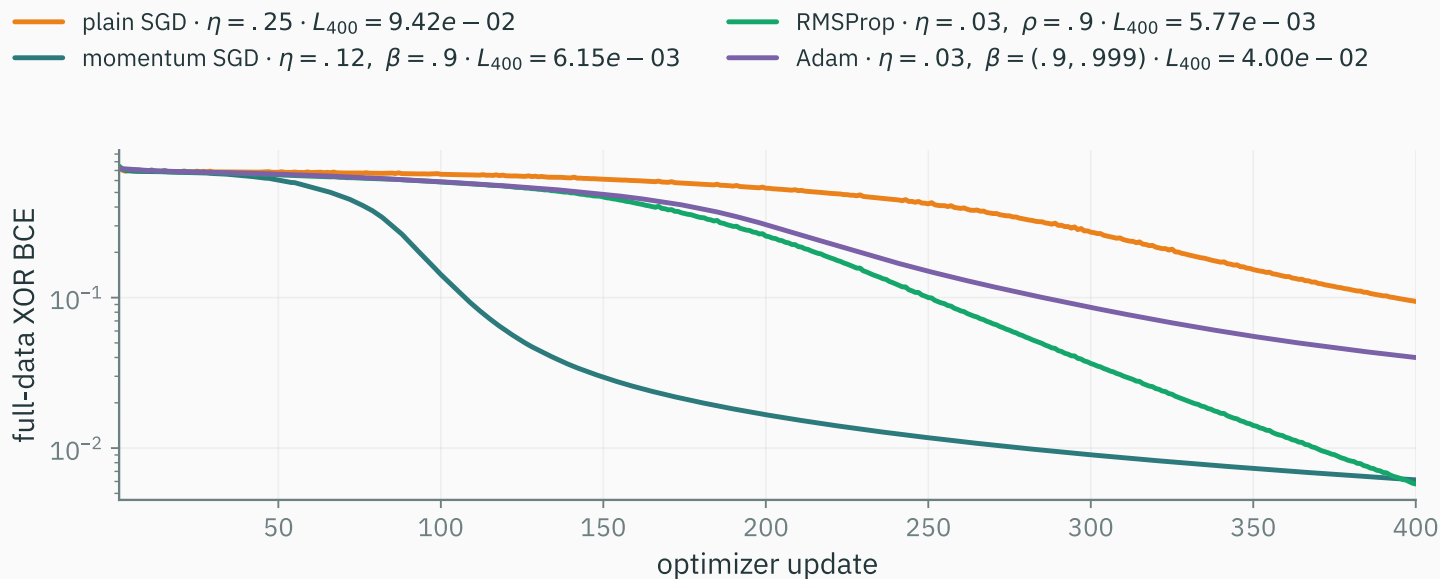
method	PyTorch configuration	learning rate
plain SGD	<code>SGD(momentum=0)</code>	0.25
momentum SGD	<code>SGD(momentum=0.9)</code>	0.12
RMSProp	<code>RMSprop(alpha=0.9)</code>	0.03
Adam	<code>Adam(betas=(.9,.999))</code>	0.03

method	PyTorch configuration	learning rate
plain SGD	<code>SGD(momentum=0)</code>	0.25
momentum SGD	<code>SGD(momentum=0.9)</code>	0.12
RMSProp	<code>RMSprop(alpha=0.9)</code>	0.03
Adam	<code>Adam(betas=(.9,.999))</code>	0.03

The rates are method-specific and were chosen so every method solves this tiny task. Their raw values are not directly comparable across update rules.

Same initialization, same 400 minibatches

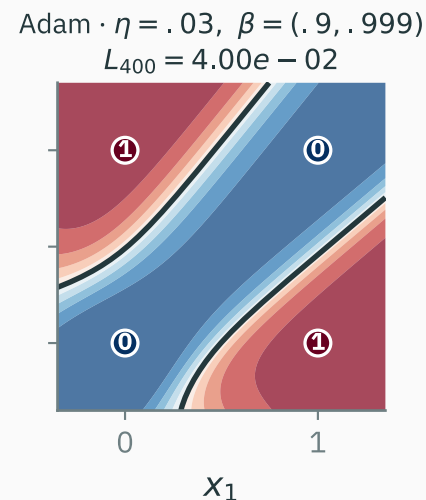
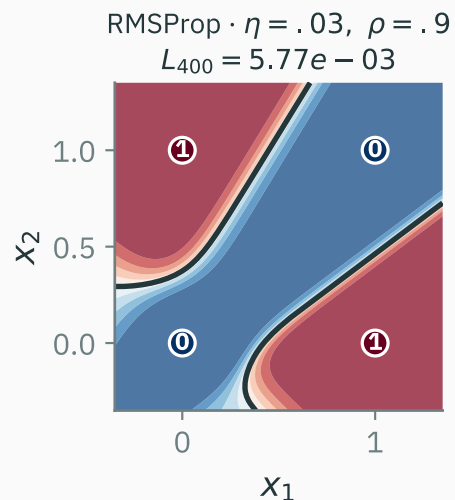
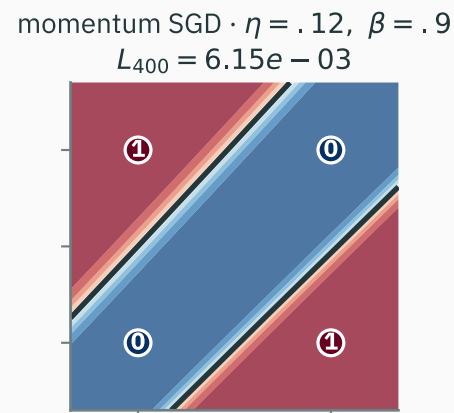
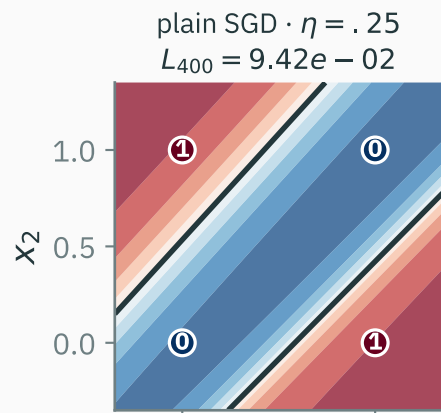
same initialization · same scheduled size-2 minibatches · 400 updates



seeded mechanism demo · seed 2026 · method-specific rates · not a benchmark or universal ranking

COMPUTED · SEED 2026 · SHARED INITIALIZATION AND SIZE-2 BATCH SCHEDULE · 400 UPDATES

Same 100% training accuracy—different learned boundaries



The PyTorch loop changes in one line

```
model.load_state_dict(initial_state) # identical start

# choose exactly one:
opt = torch.optim.SGD(model.parameters(), lr=.25)
opt = torch.optim.SGD(model.parameters(), lr=.12, momentum=.9)
opt = torch.optim.RMSprop(model.parameters(), lr=.03, alpha=.9)
opt = torch.optim.Adam(model.parameters(), lr=.03)

for indices in shared_batch_schedule:
    opt.zero_grad(set_to_none=True)
    loss = F.binary_cross_entropy_with_logits(
        model(X[indices]).squeeze(-1), y[indices])
    loss.backward(); opt.step()
```

The optimizer transforms gradients; the model, loss, data, initialization, and batch order remain controlled.

Checkpoint: what does the XOR result establish?

QUESTION

A

Adam is universally the best optimizer.

B

Momentum always needs a smaller learning rate.

C

Each stored state can be implemented in the same training loop.

D

Four training points predict large-model generalization.

Correct: C · the same backward pass can feed SGD, momentum, RMSProp, or Adam.

The controlled run shows mechanism and reproducibility. It cannot rank optimizers for other models, datasets, budgets, or validation metrics.

Notebook 04 makes every control explicit

I INTERACTIVE

CONTROLLED

Reuse one initial state and one exact 400-minibatch schedule across all four optimizers.

INSPECTABLE

Plot full-data BCE after every update and evaluate every learned decision boundary on one grid.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L05/04_xor_optimizer_lab.ipynb

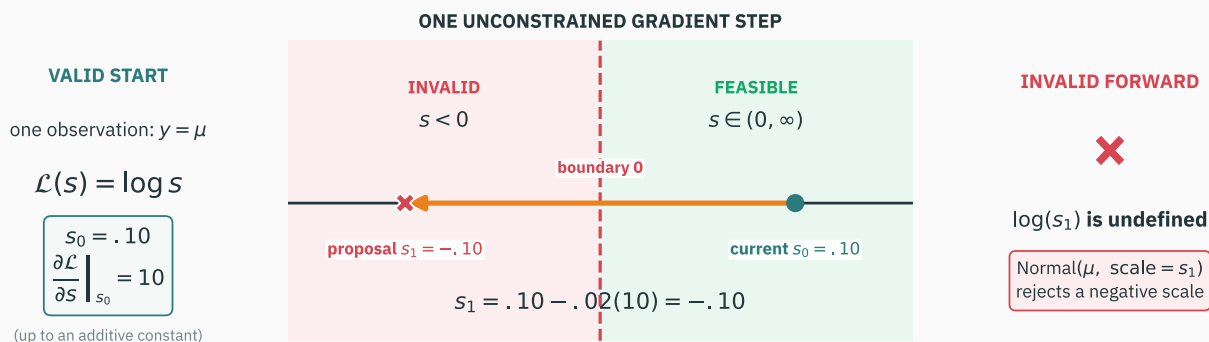
Open **XOR optimizer lab** in Colab ↗

**Constrained parameters: keep
every forward pass valid**

A valid optimizer step can produce an invalid model

A Gaussian scale must stay positive, but the optimizer sees only a gradient. For one observation with $y = \mu$,

$$\mathcal{L}(s) = \log s + \frac{(y - \mu)^2}{2s^2} = \log s, \quad s > 0.$$



$$\text{COMPUTED} \cdot s_0 = .10 \cdot \eta = .02 \cdot \text{ONE EXACT GRADIENT STEP}$$

COORDINATE CHANGE

$$r \in \mathbb{R} \rightarrow s = \text{softplus}(r) + \varepsilon > 0$$

$$\frac{\partial \mathcal{L}}{\partial r} = \frac{\partial \mathcal{L}}{\partial s} \sigma(r)$$

1. positive $(0, \infty)$: softplus or exp
2. probability $(0, 1)$: sigmoid of a logit
3. simplex interior: softmax of logits

MINIMAL PYTORCH

```
raw_scale = nn.Parameter(torch.tensor(0.0))
prob_logit = nn.Parameter(torch.tensor(0.0))
mix_logits = nn.Parameter(torch.zeros(3))
opt = torch.optim.Adam(
    [raw_scale, prob_logit, mix_logits], lr=1e-2)

opt.zero_grad(set_to_none=True)
scale = F.softplus(raw_scale) + 1e-6
prob = torch.sigmoid(prob_logit)
mix = torch.softmax(mix_logits, dim=0)
loss = objective(scale, prob, mix)
loss.backward(); opt.step()
```

Autograd applies the chain rule to the raw parameters. For Bernoulli or categorical losses, prefer logits-based functions such as BCEWithLogitsLoss and log_softmax.

CLOSED FEASIBLE SET

$$\tilde{s}_{t+1} = s_t - \eta g_t$$

$$s_{t+1} = \max(0, \tilde{s}_{t+1})$$

CHOOSE BY MEANING

Use a smooth map for a strict interior such as $s > 0$. Project when an exact boundary such as $s = 0$ is a valid model value.

MINIMAL PYTORCH

```
nonnegative = nn.Parameter(torch.tensor(2.0))
opt = torch.optim.SGD([nonnegative], lr=.25)

opt.zero_grad(set_to_none=True)
loss = 0.5 * (nonnegative + 1).square()
loss.backward()
opt.step()                                # unconstrained proposal
with torch.no_grad():
    nonnegative.clamp_(min=0.0) # projection onto [0, infinity)
```

Projection changes the parameter, but momentum or Adam state still reflects the pre-projection proposal. Inspect that interaction rather than clamping silently.

**Choose the repair that matches
the symptom**

Optimization in one view

EXAMPLES PER GRADIENT

Full batch: exact, slow update. SGD: immediate, noisy. Minibatches balance cost and variance.

DIRECTION MEMORY

Momentum averages signed gradients: cancel zigzag and preserve persistent motion.

COORDINATE SCALE

RMSProp rescales using recent squared gradients; Adam adds direction memory.

STEP SIZE OVER TIME

Tune η first. Decay late if updates prevent refinement; state the schedule clock.

VALID PARAMETERS

Reparameterize strict interiors; project only when the boundary is a legal value.

FAIR EVIDENCE

Keep initialization, data order, and update budget fixed; inspect training, validation, and state.

Start simple → diagnose one failure → add one mechanism → measure again.